Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
GRADUATION THESIS

Исследование сценариев использования и разработка системы управления Time-Series данными

Обучающийся / Student Кривоносов Егор Дмитриевич
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет программной инженерии и компьютерной техники
Группа/Group P34111
Направление подготовки/ Subject area 09.03.04 Программная инженерия
Образовательная программа / Educational program Системное и прикладное программное обеспечение 2019
Язык реализации ОП / Language of the educational program Русский
Статус ОП / Status of educational program
Квалификация/ Degree level Бакалавр
Руководитель ВКР/ Thesis supervisor Перл Иван Андреевич, кандидат технических наук, Университет ИТМО, факультет программной инженерии и компьютерной техники, доцент (квалификационная категория "ординарный доцент")

Обучающийся/Student

Документ подписан
Кривоносов Егор Дмитриевич
22.05.2023

(эл. подпись/ signature)

Кривоносов Егор Дмитриевич
(Фамилия И.О./ name and surname)

Руководитель ВКР/ Thesis supervisor

Документ подписан
Перл Иван Андреевич
22.05.2023

(эл. подпись/ signature)

Перл Иван Андреевич
(Фамилия И.О./ name and surname)

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ / OBJECTIVES FOR A GRADUATION THESIS

Обучающийся / Student Кривоносов Егор Дмитриевич
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет программной инженерии и компьютерной техники
Группа/Group P34111
Направление подготовки/ Subject area 09.03.04 Программная инженерия
Образовательная программа / Educational program Системное и прикладное программное обеспечение 2019
Язык реализации ОП / Language of the educational program Русский
Статус ОП / Status of educational program
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Исследование сценариев использования и разработка системы управления Time-Series данными
Руководитель ВКР/ Thesis supervisor Перл Иван Андреевич, кандидат технических наук, Университет ИТМО, факультет программной инженерии и компьютерной техники, доцент (квалификационная категория "ординарный доцент")

Основные вопросы, подлежащие разработке / Key issues to be analyzed
Цель работы: упрощение управления данными временных рядов, их получения, загрузки, хранения и взаимодействия с ними

Задачи:
- проанализировать существующие литературу и решения по управлению данными временных рядов;
- определить распространенные сценарии использования систем управления данными временных рядов в различных отраслях;
- изучить требования к производительности, масштабируемости и надежности этих систем, а также проанализировать, какие базы данных лучше всего использовать для хранения различных типов данных;
- изучить современные алгоритмы и структуры данных, подходящие для управления данными временных рядов и загрузки их в систему;
- спроектировать и разработать прототип системы управления данными временных рядов, отвечающий существующим требованиям;
- оценить возможности и функциональность системы, а также провести сравнение с существующими решениями.

Содержание работы:
Работа содержит четыре главы. В первой главе будут рассмотрены основные предпосылки и требования к разработке системы управления данными временных рядов. Также в

текущей главе будет проведено исследование различных сценариев использования, на основе которых уже будет разрабатываться система. В данной главе предоставлен необходимый теоретический материал для понимания практической части в следующих главах. Во второй главе подробно описан процесс выбор технологий и проектирования системы управления данными временных рядов. В третьей главе представлен процесс реализации прототипа системы, спроектированной в предыдущей главе. Последняя глава посвящена анализу полученных результатов и сравнения с аналогами.

Рекомендуемая литература:
1. Kirchgassner G., Wolters J., Hassler U. Introduction to modern time series analysis. – Springer Science & Business Media, 2012
2. Mohamudally N. (ed.). Time Series Analysis and Applications. – IntechOpen, 2018.
3. Dunning T., Friedman E. Time series databases //New Ways to Store and Access data. – 2015.¬
4. de Assuncao M. D., da Silva Veith A., Buyya R. Distributed data stream processing and edge computing: A survey on resource elasticity and future directions // Journal of Network and Computer Applications. – 2018.
5. Marjani M. et al. Big IoT data analytics: architecture, opportunities, and open research challenges // ieee access. – 2017.

Форма представления материалов ВКР / Format(s) of thesis materials:
Текстовый документ, программный код, презентация.

Дата выдачи задания / Assignment issued on: 27.09.2022

Срок представления готовой ВКР / Deadline for final edition of the thesis 30.05.2023

Характеристика темы ВКР / Description of thesis subject (topic)

Тема в области фундаментальных исследований / Subject of fundamental research: нет / not
Тема в области прикладных исследований / Subject of applied research: да / yes

СОГЛАСОВАНО / AGREED:

Руководитель ВКР/ Thesis supervisor

Документ подписан
Перл Иван Андреевич
21.05.2023

(эл. подпись)

Перл Иван Андреевич

Задание принял к исполнению/ Objectives assumed BY

Документ подписан
Кривоносов Егор Дмитриевич
21.05.2023

(эл. подпись)

Кривоносов Егор Дмитриевич

Руководитель ОП/ Head

Документ	

of educational program

подписан

Дергачев
Андрей
Михайлович

26.05.2023

(эл. подпись)

Дергачев
Андрей
Михайлович

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

АННОТАЦИЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ SUMMARY OF A GRADUATION THESIS

Обучающийся / Student Кривоносов Егор Дмитриевич
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет программной инженерии и компьютерной техники
Группа/Group P34111
Направление подготовки/ Subject area 09.03.04 Программная инженерия
Образовательная программа / Educational program Системное и прикладное программное обеспечение 2019
Язык реализации ОП / Language of the educational program Русский
Статус ОП / Status of educational program
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Исследование сценариев использования и разработка системы управления Time-Series данными
Руководитель ВКР/ Thesis supervisor Перл Иван Андреевич, кандидат технических наук, Университет ИТМО, факультет программной инженерии и компьютерной техники, доцент (квалификационная категория "ординарный доцент")

ХАРАКТЕРИСТИКА ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ DESCRIPTION OF THE GRADUATION THESIS

Цель исследования / Research goal
Целью работы является упрощение управления данными временных рядов, их получения, загрузки, хранения и взаимодействия с ними.
Задачи, решаемые в ВКР / Research tasks
Для достижения цели были установлены следующие задачи: проанализировать существующие литературу и решения по управлению данными временных рядов; определить распространенные сценарии использования систем управления данными временных рядов в различных отраслях; изучить требования к производительности, масштабируемости и надежности этих систем, а также проанализировать, какие базы данных лучше всего использовать для хранения различных типов данных; изучить современные алгоритмы и структуры данных, подходящие для управления данными временных рядов и загрузки их в систему; спроектировать и разработать прототип системы управления данными временных рядов, отвечающий существующим требованиям; оценить возможности и функциональность системы, а также провести сравнение с существующими решениями.
Краткая характеристика полученных результатов / Short summary of results/findings
Проведён анализ существующих решений для реализации систем управления данными временных рядов. Определены распространенные сценарии использования для данных

систем. На основе анализа баз данных была выявлена наиболее оптимальная для загрузки, хранения и взаимодействия с временными рядами. Спроектирован и реализован прототип системы управления данными временных рядов для аналитической платформы, написаны клиентская и серверная части, организовано клиент-серверное взаимодействие.

Обучающийся/Student

Документ подписан
Кривоносов Егор Дмитриевич
22.05.2023

(эл. подпись/ signature)

Кривоносов Егор Дмитриевич

(Фамилия И.О./ name and surname)

Руководитель ВКР/ Thesis supervisor

Документ подписан
Перл Иван Андреевич
22.05.2023

(эл. подпись/ signature)

Перл Иван Андреевич

(Фамилия И.О./ name and surname)

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ 9

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 10

ВВЕДЕНИЕ 11

1. ПРЕДПОСЫЛКИ И ТРЕБОВАНИЯ К РАЗРАБОТКЕ СИСТЕМ УПРАВЛЕНИЯ ДАННЫМИ ВРЕМЕННЫХ РЯДОВ 14

1.1 Введение во временные ряды 14

1.2 Обзор существующих систем управления данными временных рядов . 15

1.3 Исследование сценариев использования 16

1.4 Требования к системе управления данными временных рядов 18

1.5 Трудности и пробелы в исследованиях существующих систем 19

1.6 Выводы 20

2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ 21

2.1 Анализ баз данных временных рядов 21

2.2 Выбор технологии и инструментов для разработки 24

2.3 Архитектура разрабатываемого решения 26

2.4 Проектирование модели данных SQL базы 28

2.5 Методы загрузки данных 30

2.6 Выводы 32

3. РАЗРАБОТКА ПРОТОТИПА СИСТЕМЫ УПРАВЛЕНИЯ ДАННЫМИ ВРЕМЕННЫХ РЯДОВ 33

3.1 Разработка серверной части 33

3.2 Разработка клиентской части 38

3.3 Выводы 43

4. АНАЛИЗ РЕЗУЛЬТАТОВ 45

4.1 Тестирование системы 45
4.2 Сравнение с существующими решениями 46
4.3 Направления для будущих улучшений 47
ЗАКЛЮЧЕНИЕ 49
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 50
ПРИЛОЖЕНИЕ А 52
ПРИЛОЖЕНИЕ Б 54
ПРИЛОЖЕНИЕ В 57

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

БД – База Данных

JIT – Just-in-time

IoT – Internet of Things

TSM – Time-Structured Merge Tree

JS – JavaScript

JTS – Json Time Series

UI/UX – User Interface / User Experience

DPN – Data Processing Network

API – Application Programming Interface

HTTP – Hypertext Transfer Protocol

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

База данных – данные, связанные между собой, отношения которых явно выражены, при этом специально спроектированные для использования различными приложениями [1].

Облачный сервис – программное обеспечение прикладного назначения, запускаемое на удалённых вычислительных машинах и предоставляющее конечному пользователю интерфейс взаимодействия с использованием веб-технологий.

Шардинг – метод разделения и хранения единого логического набора данных в виде множества баз данных.

Репликация – механизм синхронизации содержимого нескольких копий объекта (например, содержимого базы данных).

LZ4 и ZSTD – алгоритмы сжатия данных без потерь, ориентированные на скорость сжатия и распаковки.

RabbitMQ – брокер обмена сообщениями.

Агрегация данных – процесс объединения различных строк из таблиц с помощью специальных функций, которые называются агрегаторами.

ВВЕДЕНИЕ

Быстрый рост объема данных, генерируемых различными отраслями промышленности, и возрастающая сложность современных приложений подчеркивают важность разработки эффективных и производительных систем управления данными временных рядов. Временной ряд представляет собой последовательность значений, собранных за определенные промежутки времени и расположенных в хронологическом порядке [2], является важнейшим компонентом во многих областях, таких как финансы, здравоохранение, машиностроение, исследование климата и другие. Необходимость исследования сценариев использования и разработки систем управления данными временных рядов является актуальной, поскольку эти системы играют неотъемлемую роль в облегчении анализа, хранения и доступа к данным временных рядов в различных областях.

В эпоху, когда принятие решений на основе данных становится все более распространенным, стала очевидной потребность в передовых аналитических платформах, способных эффективно обрабатывать и анализировать данные временных рядов. Такие платформы должны не только эффективно управлять данными, но и предоставлять аналитические возможности для извлечения ценных сведений и облегчения процессов принятия решений. Однако при сравнении у аналитических платформ их систем управления данными временных рядов важно учитывать такие факторы, как прием и хранение данных, производительность запросов, масштабируемость и аналитические возможности. Различные системы могут иметь разную степень поддержки этих функций в зависимости от их дизайна и вариантов использования. В совокупности выявлена проблема, что не имеется единой системы управления временными рядами в виде облачной веб-платформы, которая позволят одновременно бесплатно хранить, оптимально загружать и анализировать данные в настоящий момент.

Научная новизна этой работы заключается в комплексном исследовании сценариев использования и разработки прототипа системы управления данными временных рядов, которая отвечает специфическим требованиям этих сценариев и одновременно является фундаментом для аналитической платформы. Сочетая эффективное управление данными с аналитическими возможностями, предлагаемая система может оказать потенциальное влияние на различные отрасли, что приведет к улучшению процесса принятия решений, распределения ресурсов и прогнозирования.

Аналитический обзор источников и литературы, использованных в этом исследовании, показывает, что тема управления данными временных рядов была широко изучена [3]. Однако на проблему адаптации к различным сценариям использования, обработки постоянно растущего объема и сложности данных временных рядов, а также предоставления расширенных аналитических возможностей, можно заметить значительные возможности для инноваций в разработке соответствующих систем.

Целью данной работы является упрощение управления данными временных рядов, их получения, загрузки, хранения и взаимодействия с ними. Для достижения цели были выполнены следующие задачи:

1. Проанализировать существующую литературу и решения по управлению данными временных рядов.
2. Определить распространенные сценарии использования систем управления данными временных рядов в различных отраслях.
3. Изучить требования к производительности, масштабируемости и надежности этих систем, а также проанализировать, какие базы данных лучше всего использовать для хранения различных типов данных.
4. Изучить современные алгоритмы и структуры данных, подходящие для управления данными временных рядов и загрузки их в систему.
5. Спроектировать и разработать прототип системы управления данными временных рядов, отвечающий существующим требованиям.

6. Оценить возможности и функциональность системы, а также провести сравнение с существующими решениями.

Методы исследования, использованные в данной работе, включают сравнительный анализ, создание прототипа и экспериментальную оценку. Они направлены на решение конкретной проблемы – адаптации к различным сценариям использования, обработки растущего объема и сложности данных временных рядов, а также предоставления для управления пользовательского интерфейса.

1. ПРЕДПОСЫЛКИ И ТРЕБОВАНИЯ К РАЗРАБОТКЕ СИСТЕМ УПРАВЛЕНИЯ ДАННЫМИ ВРЕМЕННЫХ РЯДОВ

1.1 Введение во временные ряды

Временные ряды — это последовательность значений данных, записанных или собранных за регулярные промежутки времени и расположенных в хронологическом порядке [2]. Эти данные могут быть наблюдаемыми значениями, измерениями или метриками, которые представляют различные аспекты явления или процесса. Данные временных рядов характеризуются временной зависимостью, сезонностью, тенденциями и шумовыми компонентами.

Данные временных рядов широко используются в различных отраслях для таких целей, как прогнозирование, обнаружение аномалий и распознавание образов [4]. В финансовой сфере данные временных рядов широко используются для анализа цен на акции, курсов валют и экономических показателей. В здравоохранении они применяются для мониторинга пациентов, наблюдения за заболеваниями и эффективного распределения ресурсов. Инженерные приложения включают мониторинг оборудования, оптимизацию систем и диагностику неисправностей. Климатические исследования используют данные временных рядов для прогнозирования погоды и моделирования окружающей среды. Кроме того, данные временных рядов находят применение в научных исследованиях, где они могут использоваться для анализа и прогнозирования различных явлений и процессов в разных областях. Этот список лишь некоторые примеры возможных применений, и их разнообразие не ограничивается перечисленным.

1.2 Обзор существующих систем управления данными временных рядов

Аналитические платформы предоставляют инструменты и методы для обработки и анализа данных временных рядов с целью извлечения ценных сведений [5]. Эти платформы предоставляют широкий спектр инструментов для визуализации данных, статистического анализа, прогнозирования и машинного обучения [6].

Но система управления данными временных рядов в данных платформах является неотъемлемой частью, так как является основой. Аналитические платформы, такие как Grafana, Kibana и Zabbix, предлагают возможности визуализации, позволяя пользователям создавать пользовательские приборные панели, графики на основе данных временных рядов, предоставляют свои способы загрузки данных и веб-интерфейс. Важные критерии сравнения представлены в Таблице 1.

Таблица 1 – Сравнение существующих систем в аналитических платформах

	Grafana	Kibana	Zabbix
Загрузка данных из файлов	Нет	Да	Нет
Загрузка данных с других серверов	Да	Да	Да
Загрузка данных в реальном времени	Да	Нет	Да
Долгосрочное бесплатное облачное хранение данных	Нет	Нет	Нет
Визуализация данных	Да	Да	Да
Анализ данных	Ограничено	Да	Ограничено
Прогнозирование данных	Нет	Нет	Ограничено
Веб-интерфейс	Да	Да	Да

Можно заметить, что ни одна из аналитических платформ не предлагает полного спектра функциональности для работы с временными рядами. Это

обуславливает необходимость для пользователей в обращении к различным инструментам в зависимости от конкретных требований задачи.

Во-первых, загрузка данных из файлов представляется важной функцией, которую предоставляет только Kibana. Это может быть весьма удобно при обработке данных, которые уже собраны и сохранены локально.

Во-вторых, загрузка данных в реальном времени доступна в Grafana и Zabbix, но отсутствует в Kibana. Это может быть критически важно для мониторинга и анализа данных в реальном времени, особенно в динамичных системах, где наблюдаемые параметры могут быстро меняться.

В-третьих, ни одна из анализируемых платформ не предлагает долгосрочного бесплатного облачного хранения данных. Это означает, что пользователи должны самостоятельно заботиться о сохранении своих данных, либо платить за это деньги, если в платформе предусмотренная данная возможность.

В-четвертых, только Zabbix умеет производить прогнозирование данных, что говорит о том, что архитектура данных систем изначально не предполагала возможности интеграции аналитических модулей и модулей прогнозирования.

В заключение, все рассмотренные платформы предоставляют визуализацию и веб-интерфейс, что является важным условием для удобства и доступности использования.

1.3 Исследование сценариев использования

До начала проектирования нужно определить какими сценариями использования должна обладать система управления для облачной аналитической платформы sdLitica. Для этого была составлена UML-диаграмма, которая представлена на Рисунке 1.

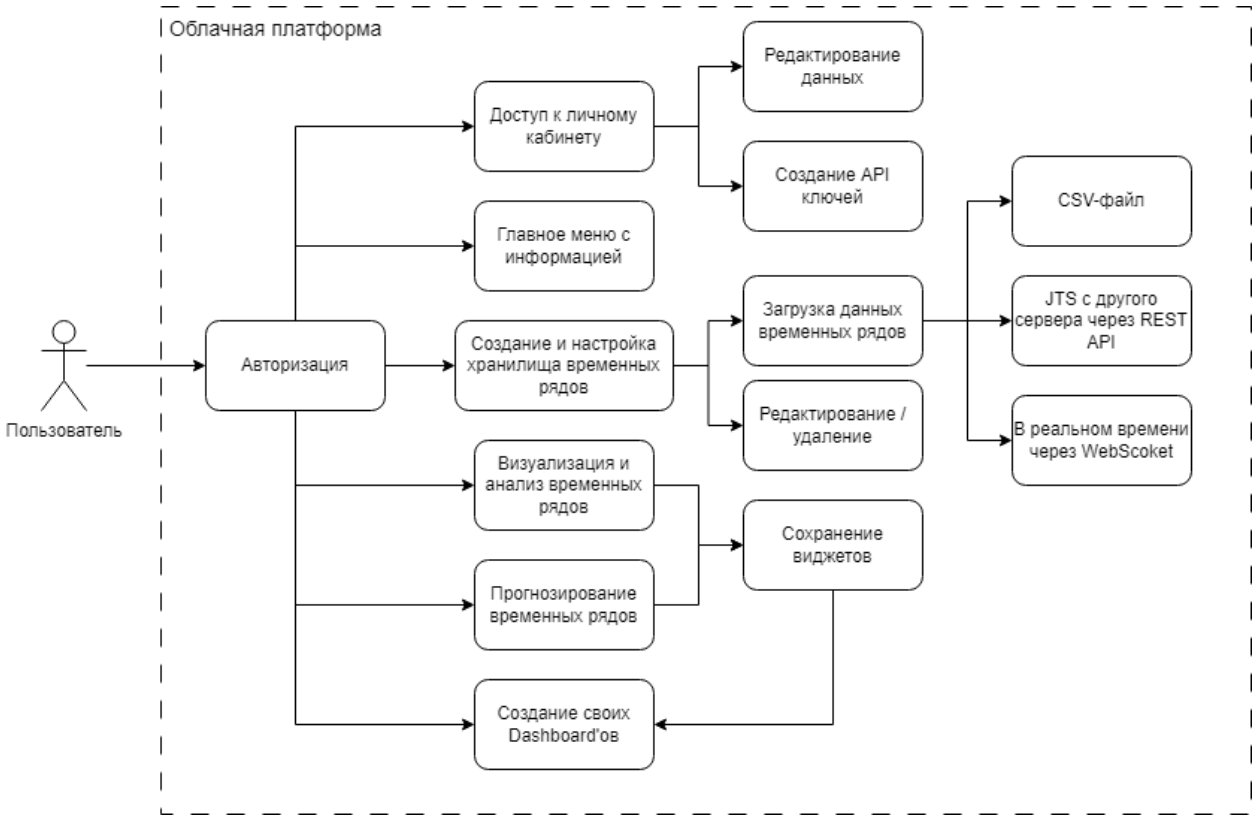


Рисунок 1 – UML-диаграмма сценариев использования облачной платформы

1. Авторизация – пользователь должен иметь возможность авторизироваться в системе с помощью своей почты и пароля, после того как он выполнит регистрацию.
2. Главное меню – при входе пользователя в систему должно отображаться главное меню с краткой инструкцией, что можно выполнять в рамках платформы и информация о системе.
3. Доступ к личному кабинету – пользователь должен иметь возможность зайти на личную страницу своего профиля, где должна отображаться информация о нем, а также иметься возможность редактирования данной информации. Ниже должны отображаться API ключи в виде таблицы и возможность их создания.

4. Создание и настройка хранилища временных рядов – пользователь должен иметь возможность создавать свое хранилище, в котором он указывает период в течение, которого хранятся данные временных рядов.
5. Загрузка данных – после создания хранилища, пользователь должен иметь возможность загрузить данные в систему тремя разными способами. Для загрузки из других источников должна иметься возможность использование API ключа и URL, который предоставляется при выборе типа загрузки с другого сервера или в реальном времени.
6. Визуализация данных – после загрузки данных, на странице визуализации пользователь должен иметь возможность строить различные графики, выводить данные в виде таблицы, смотреть статические значения и обрабатывать их. После чего сохранить в виде виджетов.
7. Создание приборной панели – пользователь должен иметь возможность создавать приборную панель и на ней отображать свои виджеты. Панель должна иметь возможность изменять размер и положение виджетов.

1.4 Требования к системе управления данными временных рядов

Различные сценарии использования часто предполагают различные требования к загрузке данных, в зависимости от частоты и объема поступающих данных. Например, для статических данных будет полезна загрузка из файла, на финансовых рынках высокочастотная торговля или данные датчиков IoT [7] может потребовать ввода данных практически в реальном времени, с чем хорошо может справиться WebSocket. А например, исследования климата, может включать периодическую пакетную обработку больших наборов данных, которые собираются из внешних источников. Понимание требований к загрузке данных для каждого случая использования

имеет решающее значение для разработки системы, которая может эффективно обрабатывать процесс ввода данных при минимизации задержек и использования ресурсов.

Масштабируемость является еще одним важнейшим аспектом систем управления данными временных рядов, поскольку она позволяет им справляться с растущими объемами данных и нагрузками на запросы. Горизонтальная масштабируемость, достигаемая с помощью распределенных архитектур, позволяет системе расширяться за счет добавления дополнительных узлов или вычислительных ресурсов [8]. Вертикальная масштабируемость, с другой стороны, предполагает увеличение мощности отдельных узлов за счет модернизации оборудования или методов оптимизации.

Безопасность для данных систем является важным требованием, поскольку передаваемые данные и хранящиеся в системе, могут быть ценным ресурсом для пользователя. Поэтому необходимо обеспечить безопасную передачу и разделение доступа между пользователями.

Дополнительно необходимо обеспечить бесплатное облачное хранение данных в системе. Для этого использовать специализированные хранилища данных временных рядов, которые позволяют эффективно работать с ними и сжимать для уменьшения объема занимаемого пространства.

1.5 Трудности и пробелы в исследованиях существующих систем

1.5.1 Обработка больших объемов данных

Существующие системы управления данными временных рядов сталкиваются с проблемами при обработке больших объемов данных, поскольку они должны обеспечивать баланс между высокой производительностью и эффективным использованием ресурсов [8].

Исследования в этой области направлены на разработку новых методов сжатия данных, индексирования и обработки запросов для решения этих проблем.

1.5.2 Загрузка данных (адаптация к различным сценариям использования)

Способность адаптироваться к различным сценариям использования, которые могут иметь различные требования к гранулярности данных, частоте обновления и задачам анализа, является значимым пробелом в исследованиях систем управления данными временных рядов [9]. Это требует разработки гибких систем, способных учитывать различные варианты использования без ущерба для производительности и функциональности.

1.6 Выводы

В данной главе представлен обзор систем управления данными временных рядов, исследование сценариев их использования, проблем и требований. Основная проблема, которая была выделена это то, что на данный момент не существует системы управления данными временных рядов в виде облачной веб-платформы, которая позволяет одновременно бесплатно хранить данные, загружать различными способами и анализировать их. В следующих главах будет рассмотрена разработка архитектуры, использованных технологий и подходов загрузки данных для системы управления данными временных рядов, которая решает эти проблемы и отвечает сценариям использования, рассмотренным в этой главе.

2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Анализ баз данных временных рядов

Базы данных временных рядов — это специализированные БД, предназначенные для хранения, извлечения и эффективного управления данными временных рядов [10]. Выбор подходящей базы данных временных рядов имеет решающее значение для эффективного управления и анализа данных временных рядов. В этом разделе проведем сравнение трех популярные базы данных временных рядов: ClickHouse, InfluxDB 2 и TimescaleDB [11]. Эти базы данных выбраны на основе их популярности, возможностей и производительности в управлении данными временных рядов.

2.1.1 Сравнение времени загрузки и эффективности хранения

Производительность загрузки данных является важнейшим аспектом любой системы управления данными временных рядов, поскольку она напрямую влияет на способность системы принимать и обрабатывать большие объемы данных. Каждая из рассматриваемой БД использует определенные алгоритмы и методы для оптимизации загрузки данных.

Сжатие данных – еще один существенный фактор при выборе базы данных временных рядов, так как оно помогает снизить требования к хранению и затраты.

ClickHouse использует столбцовый формат хранения данных и применяет такие методы, как сжатие данных и компиляция "точно в срок" (JIT) для оптимизации производительности загрузки данных. Данная БД использует различные алгоритмы сжатия, такие как LZ4 и ZSTD, для достижения высокой степени сжатия. Также у нее есть поддержка атомарной записи, которая позволяет записывать данные атомарно, что гарантирует целостность данных при возможных сбоях.

InfluxDB 2 использует специальный протокол Line Protocol, который обеспечивает быструю и эффективную загрузку данных. В частности, использование более точных значений времени (с точность до наносекунд) позволяет повысить производительность загрузки. Для оптимизации занимаемого пространства производится компрессия данных, в частности используется алгоритм GZIP для сжатия данных. Также используется собственный механизм хранения данных в виде дерева слияния временных структур (TSM) для обеспечения эффективного сжатия данных.

TimescaleDB использует преимущества существующих и проверенных методов хранения и индексирования PostgreSQL. Однако в TimescaleDB применяется новый подход к разделению данных, называемый пространственно-временным разделением, который сочетает горизонтальное разделение по временным интервалам и вертикальное разделение по пространству.

По результатам исследования представленных в статье [12] можно сделать вывод, что TimescaleDB способна импортировать данные значительно быстрее чем InfluxDB 2 и ClickHouse. Медленный прием данных InfluxDB 2 может быть связан с его файловой структурой TSM и методом сжатия. InfluxDB 2 имеет наилучшую степень сжатия 83.73%, что на 10.95% больше, чем TimescaleDB. Механизм сжатия ClickHouse имеет относительно плохую производительность, поскольку степень сжатия приближается к 100%. Это может быть связано с тем, что данная БД не оптимизирует разделение и упорядочивание данных в соответствии с меткой времени

2.1.2 Сравнение производительности запросов к данным

Часто в системах БД используют методы сжатия для уменьшения объема дискового пространства. Однако, чем больше сжатие, тем больше времени требуется на распаковку данных при выполнении запросов. Существует баланс между эффективностью хранения и скоростью чтения. Для улучшения

производительности запросов многие системы БД используют индексацию, которая обеспечивает быстрый доступ к данным. Однако, индексация может занимать дополнительное место на диске, а ее размер увеличивается вместе с размером БД. Производительность запросов имеет решающее значение при анализе данных временных рядов, поскольку она влияет на скорость извлечения информации из данных.

В Таблице 2 представлены результаты сравнения времени запросов, взятые из статьи [12], к каждой из БД. Для анализа были выбраны несколько видов запросов. Первые четыре запроса являются простыми, которые выполняются в одно действие. Последние два запроса являются комплексными и требуют проводить несколько последовательных действий для получения результата.

Таблица 2 – Производительность запросов на чтение данных из БД

Запрос	**InfluxDB 2**	**TimescaleDB**	**ClickHouse**
	Время запроса (мс)		
Средний объём торгов за 1 день	93	469	272
Средний объём торгов за 1 месяц	273	3700	1991
Нахождение самой высокой цены за неделю	146	3938	962
Нахождение самой высокой цены за месяц	440	12847	3202
Средние котировки возвращаются за день с интервалом в 5 минут	99	1614	401
Вычислите почасовое стандартное отклонение доходности средней котировки с интервалом в 5 минут в течение дня	94	1591	407

2.1.3 Заключение

На основании сравнения времени загрузки, сжатия данных и производительности запросов можно заметить, что все три базы данных - ClickHouse, InfluxDB 2 и TimescaleDB – имеют свои сильные и слабые стороны. TimescaleDB превосходит по скорости загрузки данных, в то время как InfluxDB фокусируется на сжатии и эффективности чтения данных из БД благодаря специальному языку запросов.

В контексте проектируемой системы управления данными временных рядов было принято решение выбрать InfluxDB 2 в качестве основной базы данных временных рядов из-за ее неплохой скорости загрузки данных, эффективному сжатию данных и мощному языку запросов, специально разработанному для данных временных рядов.

2.2 Выбор технологии и инструментов для разработки

Разработка эффективной системы управления данными временных рядов требует использования соответствующих технологий и инструментов, которые могут способствовать повышению производительности, масштабируемости и аналитических возможностей системы. В этом разделе мы обсудим выбранные технологии и инструменты для различных аспектов проекта.

C# является универсальным и мощным языком программирования, который предлагает сильную поддержку объектно-ориентированного программирования и имеет широкий спектр доступных библиотек и фреймворков. Фреймворк .NET является популярным выбором для разработки масштабируемых, высокопроизводительных приложений и предлагает встроенную поддержку различных баз данных и веб-сервисов для создания современных облачных систем.

InfluxDB 2 – специализированная база данных временных рядов, выбрана за ее производительность, масштабируемость и эффективные возможности хранения и доступа к данным. Механизм хранения данных и язык запросов InfluxDB оптимизированы для данных временных рядов, что делает ее подходящим выбором для данного проекта, в чем мы убедились в прошлом разделе.

MariaDB – надежная, высокопроизводительная база данных SQL, которая обеспечивает отличную совместимость с MySQL. Она выбрана за ее масштабируемость, надежность и простоту использования для управления метаданными, такими как информация о пользователях и выполняемых аналитических операциях.

Для хранения результатов анализа данных используются хранилище NoSQL, которое позволяет более гибко работать с неструктурированными данными и позволяет вмещать большие объемы данных временных рядов, генерируемых системой.

Apache Spark – распределенная вычислительная система с открытым исходным кодом, предоставляющая мощные аналитические возможности и поддержку задач машинного обучения и обработки данных. Она выбрана за масштабируемость, простоту использования и совместимость с различными системами хранения данных.

Пользовательский интерфейс системы управления данными временных рядов будет разрабатываться с использованием библиотеки React JS, которая хорошо подходит для создания интерактивных и гибких пользовательских интерфейсов.

Благодаря выбору этих технологий и инструментов проект нацелен на создание системы управления данными временных рядов, способной эффективно обрабатывать большие объемы данных, предоставлять

расширенные аналитические возможности и предлагать удобный интерфейс для визуализации данных и взаимодействия.

2.3 Архитектура разрабатываемого решения

Архитектура системы управления данными временных рядов разработана таким образом, чтобы обеспечить эффективное хранение, обработку и анализ данных, а также взаимодействие с пользователем через веб-интерфейс. На Рисунке 2 представлена разработанная архитектура для облачной аналитической платформы sdLitica.

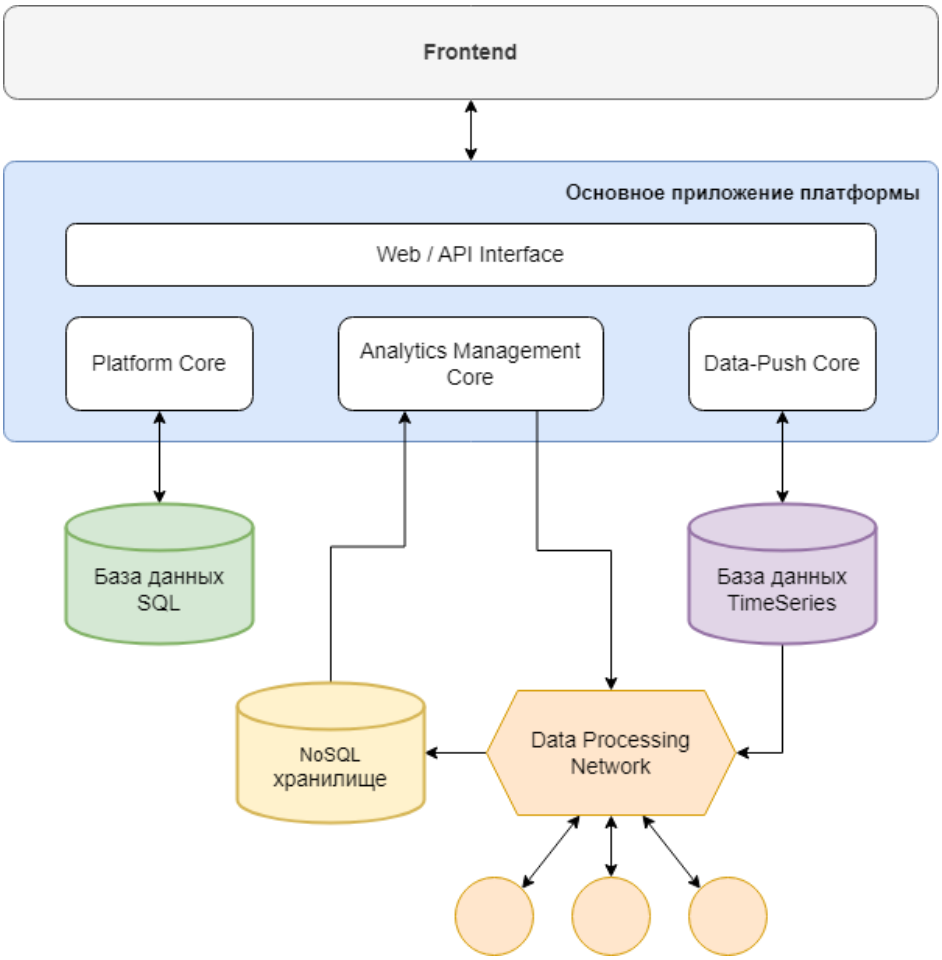


Рисунок 2 – Высокоуровневая архитектура платформы

Она состоит из нескольких взаимосвязанных компонентов:

- Platform Core – служит центральным узлом системы и включается в себя основные сервисы управления, которые осуществляют обработку таких данных как: информация о пользователе, метаданные хранилищ и временных рядов и другое.
- Analytics Management Core – предоставляет набор сервисов платформы для обработки запросов аналитик и получения результатов, фокусируется на задачах анализа данных и прогнозирования. Оно интегрируется с фреймворком Apache Spark и решениями для хранения данных NoSQL для разделения нагрузки между собой. Кроме того, очередь сообщений RabbitMQ используется для связи между основным приложением и сетью обработки данных (DPN), обеспечивая эффективную и надежную обработку данных. Данный компонент будет отвечать за анализ и прогнозирование данных, которые будут интегрированы после разработки.
- Data-Push Core – отвечает за загрузку данных временных рядов в систему. Оно поддерживает различные методы загрузки данных, такие как загрузка CSV-файлов, REST API и WebSockets. Оно обеспечивает эффективную и надежную загрузку данных, решая потенциальные проблемы, такие как несоответствие формата данных между БД и пользователем, а также проблемы с подключением к сети.
- WEB/API интерфейс – обеспечивает взаимодействие внешних систем и пользователей с системой управления данными временных рядов. Он предоставляет RESTful API для задач получения, анализа и управления данными, а также служит шлюзом для Frontend для доступа к функциям системы.

Архитектура системы управления данными временных рядов спроектирована как модульная, масштабируемая и адаптируемая, что позволяет интегрировать дополнительные компоненты или модифицировать

существующие для удовлетворения меняющихся требований и сценариев использования.

2.4 Проектирование модели данных SQL базы

Сохранение метаданных в БД служит важнейшей частью системы управления данными временных рядов, храня и управляя метаданными, связанными с пользователями, хранилищами, временными рядами, приборными панелями, виджетами и аналитическими модулями. Модель данных разработана для оптимизации поиска и управления данными при обеспечении результативного и организованного хранения. Инфологическая модель представлена на Рисунке 3.

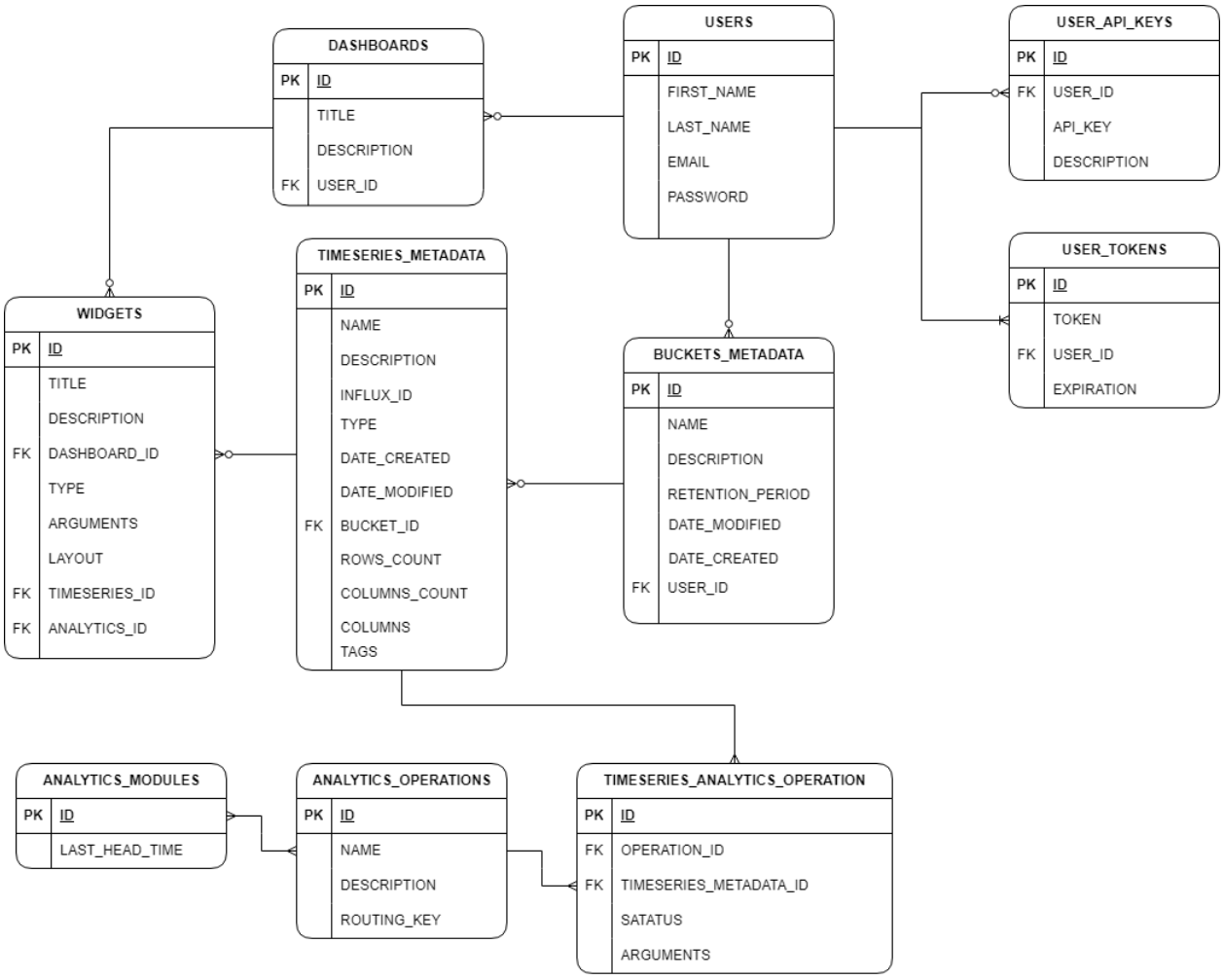


Рисунок 3 – Инфологическая модель базы данных

База данных содержит:

- Таблица USERS отвечает за хранение информации о зарегистрированных пользователях, что позволяет системе управлять учетными записями пользователей и аутентифицировать их при доступе к системе.
- Таблица USER_TOKENS управляет маркерами аутентификации и сроками их действия, что позволяет системе проверять права доступа пользователей и поддерживать соединение пользователя с системой.
- Таблица USER_API_KEYS связывает пользователей с их API-ключами, обеспечивая безопасный программный доступ к функциям системы, таким как загрузка данных. Каждый API-ключ связан с пользователем и содержит описание его назначения.
- Таблица BUCKETS_METADATA содержит информацию о хранилищах в InfluxDB, которые являются логической единицей для организации данных временных рядов. Основная задача данного хранилища содержать данные только за определенный период, указанный пользователем, для уменьшения затрат на занимаемое пространство.
- Таблица TIMESERIES_METADATA управляет метаданными, относящимися к отдельным временным рядам, позволяя системе отслеживать такую важную информацию, как название временного ряда, описание, тип, идентификатор в InfluxDB, дата создания, дата изменения, идентификатор хранилища, количество строк и столбцов, а также названия столбцов.
- Таблица DASHBOARDS отвечает за поддержание созданных пользователем приборных панелей, которые отображают визуализацию и аналитики данных временных рядов.

- В таблице WIDGETS хранится информация о виджетах, связанных с приборными панелями, которые представляют собой отдельные компоненты, отображающие конкретные визуализации или аналитические результаты.
- Таблица ANALYTICS_MODULES отслеживает последнее время обращения для каждого аналитического модуля, гарантируя, что система может отслеживать последние операции, обработанные этими модулями.
- В таблице ANALYTICS_OPERATIONS хранится информация об операциях аналитики, включая их название, описание и ключ маршрутизации RabbitMQ.
- Таблица TIMESERIES_ANALYTICS_OPERATION связывает аналитические операции с метаданными временного ряда и хранит информацию об их статусе и переданных аргументах, позволяя системе отслеживать текущие аналитические задачи и связанные с ними данные временного ряда.

Для обеспечения эффективного поиска и управления данными в БД применяются соответствующие стратегии индексирования. Индексы созданы на столбцах первичных и внешних ключей, а также на столбцах, часто используемых в поисковых запросах или фильтрах. Это поможет ускорить работу БД и позволит системе управления данными временных рядов быстро взаимодействовать с большие объемами метаданных.

2.5 Методы загрузки данных

Различные сценарии использования могут потребовать различных методов загрузки данных для оптимизации производительности и функциональности. В этом разделе рассмотрим три метода загрузки данных и

их потенциальные сценарии использования, которые реализуются в рамках разрабатываемой системы.

Загрузка из файлов – этот метод предполагает загрузку данных из CSV-файлов непосредственно в базу данных временных рядов. Этот подход используется для ситуаций, когда пользователи имеют большую партию исторических данных временных рядов, сохраненных в формате CSV, и им необходимо импортировать их в систему. Это также удобный метод для пользователей, которые хотят перенести данные из других систем или БД, поддерживающих экспорт CSV.

Загрузка REST API (JSON Time Series или JTS) – этот метод использует REST API для получения данных с других серверов и импорта их в систему. JSON Time Series (JTS) — это формат данных, разработанный специально для данных временных рядов. Этот метод подходит для интеграции со сторонними сервисами и приложениями, которые предоставляют свои данные через API, позволяя пользователям импортировать и анализировать данные временных рядов из различных источников.

Потоковая передача данных через WebSockets – этот метод предполагает получение потоков данных в реальном времени через WebSockets и хранение данных непосредственно в базе данных временных рядов. Этот подход подходит для сценариев, в которых данные генерируются и передаются в реальном времени, например устройства IoT, сенсорные сети или системы мониторинга. Использование WebSockets обеспечивает низкую задержку при получении данных и позволяет системе обрабатывать по мере их поступления.

Каждый из этих методов загрузки данных имеет свои преимущества и подходит для конкретных случаев использования. Выбор метода зависит от требований и ограничений источника данных пользователя.

2.6 Выводы

В текущей главе были проанализированы различные аспекты проектирования системы управления данными временных рядов. Было проведено сравнение и выбор наиболее подходящих технологий, инструментов и БД для конкретных требований. Также представили подробный архитектурный проект системы, который включает в себя ее основные компоненты и их взаимодействие. Кроме того, обсудили модель данных SQL БД и различные методы загрузки данных, которые удовлетворяют различным сценариям использования. Учитывая эти факторы, была заложена основа для разработки целостного прототипа системы управления данными временных рядов, который служит основой для аналитической платформы.

3. РАЗРАБОТКА ПРОТОТИПА СИСТЕМЫ УПРАВЛЕНИЯ ДАННЫМИ ВРЕМЕННЫХ РЯДОВ

3.1 Разработка серверной части

Архитектура серверной части системы управления данными временных рядов основана на компонентном подходе. Такой способ проектирования позволяет разделить задачи, что облегчает обслуживание, тестирование и возможное расширение в будущем. В основном построена вокруг следующих компонентов: Platform Core, Data-Push Core, Web / API Interface, а также необходимые компоненты для обеспечения производительности, масштабируемости и безопасности системы.

3.1.1 Разработка компонента Platform Core

Компонент Platform Core отвечает в основном за управление данными временных рядов, хранящимися в базе данных InfluxDB, а также метаданными, хранящимися в MariaDB. Разработка этого компонента была в основном сосредоточена на обработке и организации данных, обеспечении целостности данных и оптимизации операций хранения и поиска данных, ключевые элементы реализации компонента представлена в Приложении А.

Управление пользователями – отвечает за регистрацию, вход и аутентификацию пользователей. Во время регистрации такие данные пользователя, как имя, электронная почта и зашифрованный пароль, собираются и хранятся в таблице USERS в MariaDB. Когда пользователь входит в систему, система сверяет предоставленные им учетные данные с хранящейся информацией. После успешной аутентификации генерируется токен и сохраняется в таблице USER_TOKENS, который используется для последующих аутентифицированных запросов. Также пользователь может создавать API ключ для получения доступа из внешний ресурсов к нашей системе. Данная возможность была добавлена из-за того, что в нашей системе присутствует загрузка данных через REST API и WebSocket, которые требуют

дополнительной авторизации для предоставления возможности выполнения данных действий.

Управление информационными панелями – пользователь может создавать свои собственные информационные панели для визуализации данных. Каждый виджет на панели предоставляет собой объект информации, на которой могут быть выведены графики или аналитические данные. Для самих виджетов важными данными являются к какой аналитике или временном ряду они привязаны, аргументы для дополнительной обработки и расположение на клиентской части.

Управление хранилищами – каждое хранилище представляет собой логическую единицу, которая связывает хранимые в ней временные ряди и период времени, в течение которого записи будут храниться в системе, данная информация хранится в таблице BUCKETS_METADATA. После истечения срока происходит автоматическое удаление записей, которые превысили срок хранения и обновление метаданных. Период в течение, которого данные будут храниться, настраивает сам пользователь при создании хранилища.

Управление временными рядами – система позволяет пользователям загружать данные временных рядов в различных форматах, включая файлы CSV, JTS и потоки данных WebSocket. Загруженные данные хранятся в InfluxDB 2.7. Для каждого загруженного временного ряда собираются метаданные, которые хранятся в таблице TIMESERIES_METADATA. Их сбор необходим для того, чтобы уменьшить количество обращений к InfluxDB. Тем самым система для получения общей информации обращается к SQL базе, а если ей требуется получение самих данных, обращение идет к БД временных рядов.

Была разработана возможность производить обработку данных пользователем. Эта функция полезна, когда временной ряд насчитывает более 100 тысяч записей. Пользователь может выбрать период, за который ему нужно

получить данные, также их можно разделить на промежутки и выполнить агрегацию данных с помощью различных функций, таких как: mean, median, last, min, max, sum, stddev и другие. Тем самым происходит уменьшение количества точек, которое запрашивает пользователь и увеличение производительности получения данных.

При получении сами данных из InfluxDB была выявлена проблема, что несколько метрик группируются в одно поле Field. Это значит, что для каждой метрики строится свой временной ряд (таблица результата). Если бы InfluxDB формировал JSON-файл без дополнительной обработки, который возвращается обратно на клиентскую часть у нас, размер получался бы K × N метрик, где K – количество метрик, N – количество записей. Для уменьшения размера получаемого файла, перед отправкой происходит сбор всех метрик в единый объект временного ряда и только затем происходит отправка данных обратно. Тем самым было получено, что размер JSON-файла возвращаемого становится в K раз меньше.

3.1.2 Разработка компонента Data-Push Core

Загрузка данных является важнейшей частью системы, обеспечивая плавный и эффективный ввод данных временных рядов в платформу. Данный компонент позволяет решить проблему совместимости форматов, получаемых от пользователя и принимаемого формата БД временных рядов, ключевые элементы реализации компонента представлена в Приложении Б.

Для файлов CSV был разработан собственный парсер, который считывает данные, проверяет формат, а затем разбивает данные на отдельные записи временного ряда. Этот процесс включает в себя разделение данных на поля и теги. Поля содержат фактические данные измерений, а теги используются для индексации и фильтрации данных. Теги всегда начинаются

с символа подчеркивания и всегда являются строкой. Пример формата CSV-файла представлен в Таблице 3.

Таблица 3 – Пример формата CSV-файла

```
time,x,y,_cpuk
2023-04-29T14:41:15.000Z,2,3,three
2023-04-29T15:41:15.001Z,2,4,three
2023-04-29T16:41:15.002Z,2,5,three
2023-04-29T17:41:15.003Z,3,3,three
2023-04-29T18:41:15.004Z,3,4,three
2023-04-29T18:42:15.005Z,2,1,three
2023-04-29T19:13:15.006Z,3,2,three
```

Для данных, поступающих из REST API или потоков данных WebSocket, были разработаны специальные адаптеры. Адаптеры преобразуют входящие данные в стандартный формат, понятный базе данных временных рядов, а затем загружают их в InflxuDB. Пример формата JSON-файлов для REST API и WebSocket представлены в Таблице 4 [13] и Таблице 5. Выбор данного формата для WebSocket обусловлен тем, что операции по примитивному массиву выполняются быстрее, чем по массиву объектов и размер самих передаваемых данных представляются в достаточно компактной форме, что уменьшает размер передаваемого файла.

Таблица 4 – Пример формата Json Time Series файла

```
{
  "docType": "jts",
  "version": "1.0",
  "header": {
    "startTime": "2023-04-16T02:00:00.000Z",
    "endTime": "2023-04-16T02:20:43.000Z",
    "recordCount": 1,
    "columns": {
      "0": {
        "name": "Temperature",
        "dataType": "NUMBER",
        "renderType": "VALUE",
        "format": "0.###"
```

```
        }
      }
    },
    "data": [
      {
        "ts": "2023-04-16T02:00:39.000Z",
        "f": { "0": {"v": 99} }
      }
    ]
```

Таблица 5 – Пример формата JSON-файла для WebSocket

```
{
  "time": ["2023-05-10T02:05:40.000Z", "2023-05-10T02:15:42.000Z"],
  "fields": {
    "field1": [14323, 14324245]
  },
  "tags": {
    "_tag1": ["three", "three"]
  }
}
```

Каждый из этих способов загрузки требует API ключ для авторизации, которые передаются в заголовке. Заголовок представляется вида: apiKeyToken <ключ>. Дополнительно для веб-сокета была добавлена обработка, когда пользователь забыл закрыть соединение после завершения передачи данных. Это может быть, быть вызвано из-за сбоя соединения или забытой строчки ws.close(). В таком случае автоматически завершается сессия и пользователь сможет повторно подключиться, если ему это понадобится.

При разработке компонента загрузки данных были продуманы механизмы проверки данных и обработки ошибок. Когда данные загружаются в систему, они проверяются на правильность формата и полноту. Любые ошибки, возникающие в процессе загрузки данных, такие как отсутствие данных или неправильный формат, фиксируются и сообщаются пользователю, что обеспечивает сохранение целостности данных в системе.

3.1.3 Реализация мер безопасности

Безопасность системы была обеспечена путем внедрения целого ряда мер. К ним относятся надежная аутентификация пользователей, контроль доступа, шифрование данных. Эти меры безопасности были настроены с использованием возможностей, предоставляемых фреймворком ASP.NET Core, на котором построена система.

Аутентификация пользователей: в системе реализована надежная аутентификация пользователей с использованием фреймворка Identity, предоставляемого ASP.NET Core. Когда пользователь регистрируется или входит в систему, его учетные данные сверяются с данными пользователя, хранящимися в MariaDB. Пароли хранятся в хешированном формате через SHA256 для обеспечения их безопасности.

Шифрование данных: для защиты конфиденциальных данных в системе реализовано шифрование данных при передаче. При передаче все соединения между клиентом и сервером шифруются с помощью SSL/TLS, что гарантирует, что любые данные, передаваемые по сети, не могут быть перехвачены и прочитаны неавторизованными лицами.

3.2 Разработка клиентской части

Клиентская часть приложения, которая формирует пользовательский интерфейс (UI), была разработана с использованием React, популярной библиотеки JavaScript для создания пользовательских интерфейсов. React был выбран за его скорость, масштабируемость и компонентную архитектуру, которая позволяет лучше организовать код и повторно использовать его. Архитектура клиентской стороны разработана таким образом, чтобы обеспечить реализацию сценариев использования, выявленных ранее.

Веб-приложение на стороне клиента отвечает за представление данных временного ряда пользователю, позволяет ему взаимодействовать с данными и отправляет действия пользователя на сервер для обработки.

3.2.1 Разработка пользовательского интерфейса

Интерфейс включает экраны для аутентификации пользователя, визуализации данных временного ряда, управления данными (включая загрузку и выгрузку данных) и настройки системы. Каждый экран состоит из нескольких компонентов React, которые организованы в иерархическом порядке. Иллюстрации разработанного пользовательского интерфейса представлены в Приложении В.

Была разработана страница входа в систему (Рисунок 4) и создания нового аккаунта. Данная страница имеет небольшую форму, где вводятся почта пользователя и его пароль. Каждое поле формы имеет валидацию на вводимые значения.

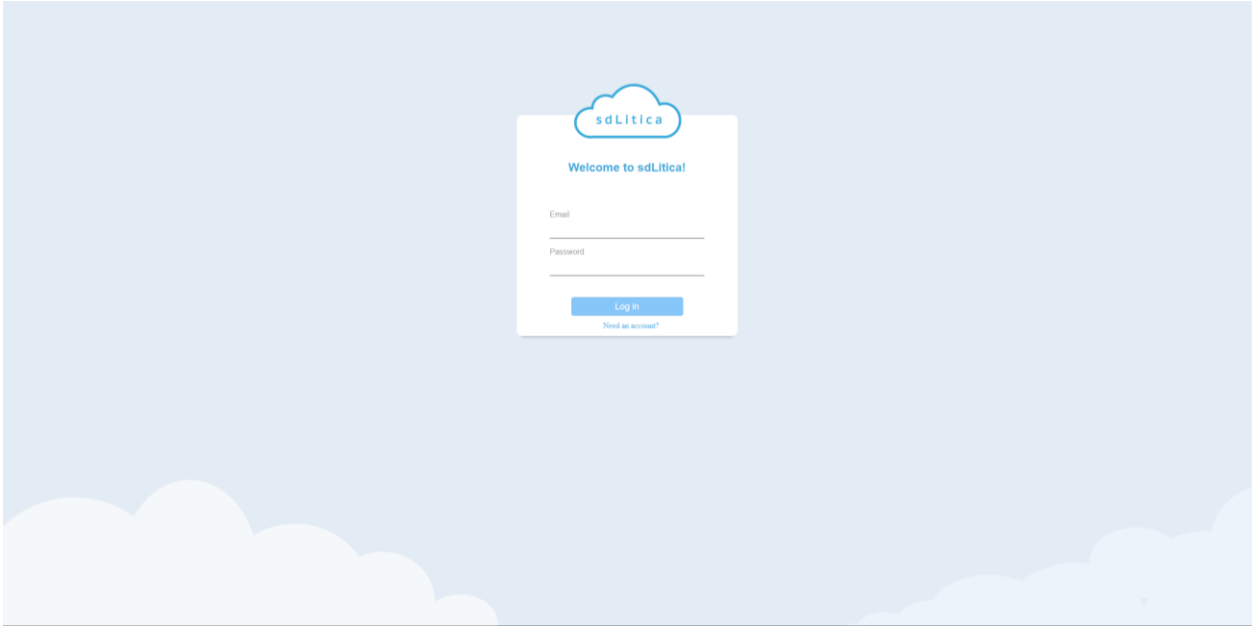


Рисунок 4 – Страница аутентификации

Для каждой страницы был разработан общий макет представления. Он включается в себя шапку страницы с отображением пользователя, боковую панель с навигацией по страницам и основную часть, где происходит отображение содержимого каждой страницы.

В разделе личного кабинета пользователя представлена информацию о нем и возможность её изменения. Ниже располагается таблица с API ключами пользователя и их описанием, а также поле для создания нового ключа.

В разделе загрузки данных временных рядов пользователь может выбрать один из трех видов загрузки. После заполнения всех форм получить на выходе уже загруженный временной ряд, если данные загружаются непосредственно через CSV-файл или API ключ и URL для выполнения загрузки из внешний ресурсов.

В разделе хранилищ представлена о них информация, можно узнать о периоде хранения данных временных рядов, создать новые, изменить настройки, удалить. Также на данной странице имеется сортировка и фильтрация, для быстрого поиска нужных. Перейдя внутрь, открывается страница временных рядов, где они отображаются в виде таблицы (Рисунок 5). Сами временные ряды можно раскрыть и посмотреть такую информацию как описание, дату создания, дату изменения, размер, количество строк, количество столбцов и возможно экспорта в виде CSV-файла. Данная страница аналогична хранилищам поддерживает фильтрацию и сортировку. Также имеется кнопка View, которая позволяет перейти к визуализации их.

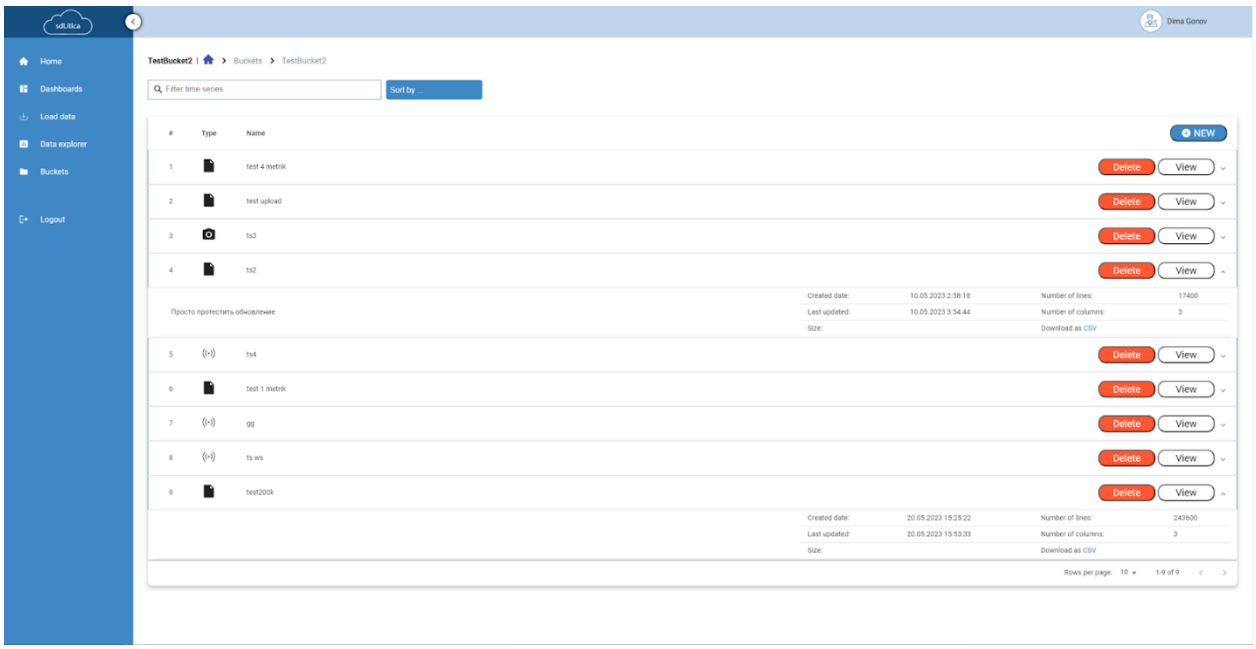


Рисунок 5 – Страница временных рядов

3.2.2 Визуализация данных

Одним из ключевых аспектов веб-приложения на стороне клиента является возможность визуализации данных временных рядов. Для этого использовалась библиотека визуализации данных, совместимые с React, такая как Plotly.js.

Система предоставляет несколько типов визуализации данных, включая линейные графики, гистограммы, диаграммы и таблицы. Выбор визуализации может быть настроен пользователем в зависимости от его потребностей. Кроме того, приложение позволяет пользователям взаимодействовать с визуализацией, например, увеличивать масштаб на определенных временных периодах или наводить курсор на точки данных для просмотра подробной информации. Также доступна фильтрация данных для отображения и конвертирование их по определенным периодам через функции агрегации. Пример визуализации представлен на Рисунке 6.



Рисунок 6 – Пример визуализации временных рядов

Графики, которые были визуализированы, можно сохранить в виде виджета, а затем отображать на приборных панелях, которые были созданы пользователем. Виджеты можно располагать в любом порядке на панели, также они поддерживают возможность увеличения и уменьшения размера. После того как пользователь закончил настройку панели он может её сохранить и смотреть изменения во временных рядах. Пример визуализации виджетов на приборной панели представлен на Рисунке 7.

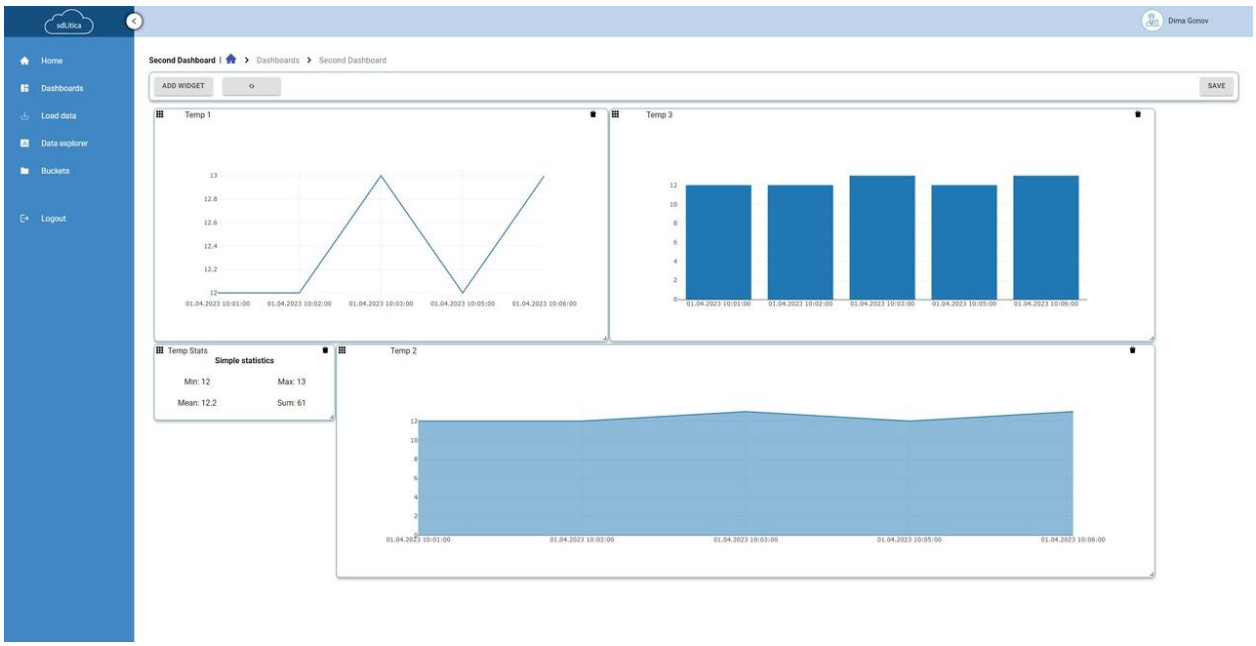


Рисунок 7 – Пример визуализации виджетов

3.2.3 Интеграция с серверной частью

Клиентское приложение взаимодействует с серверной частью через RESTful API. Этот API предоставляет различные конечные точки, которые клиентская сторона может вызывать для выполнения таких действий, как получение данных временного ряда, загрузка новых данных и обновление настроек.

Клиентская сторона использует HTTP-клиент Axios, для выполнения HTTP-запросов к серверу. Ответы от сервера обычно поступают в формате JSON, который затем обрабатывается на стороне клиента для обновления пользовательского интерфейса.

3.3 Выводы

В данной главе был изложен процесс разработки прототипа спроектированной системы управления данными временных рядов. Была разработана серверная и клиентская часть системы. Рассмотрены основные

особенности построения компонентов серверной части с учетом эффективности, производительности и безопасности инфраструктуры с использованием C#, .NET Core и других технологий. Также спроектирован пользовательский интерфейс с помощью React, который позволяет взаимодействовать с системой и данными временных рядов.

Успешное завершение этого этапа разработки закладывает основу для следующего этапа, который включает в себя тестирование и сравнение с аналогами данных систем.

4. АНАЛИЗ РЕЗУЛЬТАТОВ

4.1 Тестирование системы

После того, как была закончена разработка, было проведено ручное тестирование системы. В ходе, которого были успешно проверенные все способы загрузки данных, взаимодействие с пользовательским интерфейсом по сценариям использования, работа интерфейсов визуализации и собраны некоторые метрики.

В ходе тестирование были получены результаты по размерам получаемых файлов с данными для визуализации от sdLitica и InfluxDB. Как было замечено ранее во время разработки, из-за проблемы с извлечением данных из InfluxDB файл получается большого размера из-за составления для каждой метрики новых таблиц и переизбытка данных. Как можно заметить по результатам отраженных в Таблице 6, Таблице 7 и Таблице 8. sdLitica уменьшает размер файла из-за сбора всех метрик для временного промежутка в один объект. Размеры файлов представлены в Кбайт.

Таблица 6 – Сравнение размеров файлов при 1 метрике

Кол-во строк	sdLitica	InfluxDB	Отношение
600	46,8	72,0	1,5
4800	372,6	574,2	1,5
17000	1350,1	2220,0	1,6
243000	18899,7	30836,4	1,6

Таблица 7 – Сравнение размеров файлов при 2 метриках

Кол-во строк	sdLitica	InfluxDB	Отношение
600	50,7	143,8	2,8
4800	404,1	1148,1	2,8
17000	1464,2	4439,6	3,0
243000	20497,7	61666,1	3,0

Таблица 8 – Сравнение размеров файлов при 4 метриках

Кол-во строк	sdLitica	InfluxDB	Отношение
600	58,6	287,2	4,9
4800	467,2	2295,9	4,9
17000	1692,9	8879,1	5,2
243000	23700,1	123331,3	5,2

Также были измерены значения сжатия данных с использованием InfluxDB в качестве БД временных рядов. По результатам было получено, что степень сжатия данных варьируется от 10% до 50%. Это связано с тем, что в базе могут быть разные типы данных, корреляции между значениями, количества данных и другими факторами.

4.2 Сравнение с существующими решениями

В области систем управления данными временных рядов уже есть несколько устоявшихся решений у аналитических платформ, таких как Grafana, Kibana и Zabbix. Эти системы были широко распространены для различных случаев использования, и они служат полезным эталоном для оценки разработанной системы для платформы sdLitica. Был проведен сравнительный анализ этих систем и результаты сравнения представлены в Таблице 9.

Таблица 9 – Сравнение существующих систем с sdLitica

	Grafana	Kibana	Zabbix	sdLitica
Загрузка данных из файлов	Нет	Да	Нет	Да
Загрузка данных с других серверов	Да	Да	Да	Да
Загрузка данных в реальном времени	Да	Нет	Да	Да

Долгосрочное бесплатное облачное хранение данных	Нет	Нет	Нет	Да
Визуализация данных	Да	Да	Да	Да
Анализ данных	Ограничено	Да	Ограничено	В процессе интеграции
Прогнозирование данных	Нет	Нет	Ограничено	В процессе интеграции
Веб-интерфейс	Да	Да	Да	Да

Разработанный прототип системы, как указано в последней колонке, включает в себя все выделенные функциональные возможности при сравнении систем. Он предоставляет возможности для загрузки данных из файлов и с внешних ресурсов, а также для загрузки данных в режиме реального времени. Он также поддерживает долгосрочное хранение данных в облаке бесплатно, чего не предлагают остальные системы.

С точки зрения визуализации данных разработанный прототип системы соответствует возможностям своих аналогов. Однако он превосходит Zabbix по возможностям анализа данных, которые в настоящее время интегрируются для предоставления более продвинутых функций. Возможности системы по прогнозированию данных также находятся в стадии интеграции и нацелены на предоставление функциональности, недоступной в Grafana или Kibana. Наконец, как и другие системы, sdLitica предоставляет веб-интерфейс для легкого доступа и навигации.

4.3 Направления для будущих улучшений

Для расширения возможностей системы и улучшения пользовательского опыта будущие разработки будут направлены на интеграцию уже разработанных функций анализа данных и прогнозирования. Это предоставит пользователям более мощные инструменты для извлечения информации из

данных временных рядов. Дальнейшие усовершенствования могут также включать расширение совместимости системы с различными источниками данных и интеграцию большего количества сторонних приложений для повышения универсальности. А также добавления поддержки других форматов данных, таких как XML, Parquet и Pickle.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы был разработан прототип системы для упрощения управления данными временных рядов, который включает в себя получение, загрузку, хранение и взаимодействие с ними. Сравнительный анализ существующих систем помог определить ключевые особенности, необходимые для разработки системы управления данными временных рядов. Исследование также выявило общие сценарии использования этих систем в различных отраслях, которые легли в основу дизайна и функциональности разработанной системы.

Далее, в ходе исследования были изучены требования к производительности, масштабируемости и надежности систем управления данными временных рядов, особое внимание было уделено выбору баз данных, наиболее подходящих для хранения различных типов данных. Это исследование привело к решению использовать комбинацию InfluxDB для данных временных рядов, MariaDB для метаданных.

Финальным этапом данной работы стало проектирование и разработка прототипа системы управления данными временных рядов для аналитической платформы sdLitica, которая отвечает современным требованиям отрасли. Возможности и функциональность sdLitica были оценены и сравнены с существующими решениями. Хотя в настоящее время в системе отсутствуют возможности глубокого анализа данных и прогнозирования, была заложена основа для интеграции этих функций в будущем. Можно сделать вывод, что sdLitica с ее полным и сбалансированным набором функций, эффективным управлением данными и пользовательским интерфейсом является перспективным решением для управления данными временных рядов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Столлингс В. Операционные системы: внутренняя структура и принципы проектирования. 9-е изд. //СПб.: ООО "Диалектика", 2020. – 1264 с.
2. Kirchgässner G., Wolters J., Hassler U. Introduction to modern time series analysis. – Springer Science & Business Media, 2012. – 1 с.
3. Dunning T., Friedman E. Time series databases //New Ways to Store and Access data. – 2015.
4. Mohamudally N. (ed.). Time Series Analysis and Applications. – IntechOpen, 2018.
5. Mishra N. et al. A comprehensive survey of data mining techniques on time series data for rainfall prediction //Journal of Ict Research & Applications. – 2017. – Т. 11. – №. 2. – 170-178 с.
6. Nielsen A. Practical time series analysis: Prediction with statistics and machine learning. – O'Reilly Media, 2019.
7. Marjani M. et al. Big IoT data analytics: architecture, opportunities, and open research challenges //ieee access. – 2017. – Т. 5. – С. 5247-5261.
8. de Assuncao M. D., da Silva Veith A., Buyya R. Distributed data stream processing and edge computing: A survey on resource elasticity and future directions //Journal of Network and Computer Applications. – 2018. – Т. 103. – С. 1-17.
9. Jensen S. K., Pedersen T. B., Thomsen C. Time series management systems: A survey //IEEE Transactions on Knowledge and Data Engineering. – 2017. – Т. 29. – №. 11. – С. 2581-2600.
10. Liu R., Yuan J. Benchmarking time series databases with IoTDB-benchmark for IoT scenarios //arXiv preprint arXiv:1901.08304. – 2019. – 1 с.

11. DB-Engines Ranking of Time Series DBMS [Электронный ресурс]. – URL: https://db-engines.com/en/ranking/time+series+dbms (дата обращения 19.03.2023)

12. Barez F., Bilokon P., Xiong R. Benchmarking Specialized Databases for High-frequency Data // arXiv preprint arXiv:2301.12561. – 2023.

13. JSON Time Series [Электронный ресурс]. — Режим доступа: https://docs.eagle.io/en/latest/reference/historic/jts.html (дата обращения: 22.03.23)

ПРИЛОЖЕНИЕ А

Листинг 1 – Пример контроллера для создания API ключа

```
[HttpPost("apikeys")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
public ApiEntityPage<UserApiKeyJsonEntity> CreateApiKey([FromBody]
IDictionary<string, string> requestBody)
{
    string keyDescription = requestBody.ContainsKey("description") ?
requestBody["description"] : "not set";

    UserApiKey apiKey = _userService.CreateUserApiKey(UserId,
keyDescription);

    ApiEntityPage<UserApiKeyJsonEntity> result = new
ApiEntityPage<UserApiKeyJsonEntity>(new UserApiKeyJsonEntity(apiKey),
Request.Path.ToString());

    Response.StatusCode = StatusCodes.Status201Created;

    return result;
}
```

Листинг 2 – Пример создания и обновления токена пользователя

```
public async Task<UserToken> GetNewTokenAsync(User user)
{
    if (user == null) throw new ArgumentNullException(nameof(user));

    UserToken userToken = _userTokenRepository.GetByUser(user);
    if (userToken == null)
    {
        userToken = UserToken.Create(user,
_appSettings.TokenExpirationInHours);
        _userTokenRepository.Add(userToken);
    }
    else
    {
        userToken.CreateToken(_appSettings.TokenExpirationInHours);
        _userTokenRepository.Update(userToken);
        }
    await _userTokenRepository.SaveChangesAsync();
    return userToken;
}
```

Листинг 3 – Пример извлечения и обработки данных

```
public async Task<List<FluxTable>> ReadMeasurementById(string measurementId,
string bucketId, string from, string to, string step, string agrFn)
{
    var rangeStart = !string.IsNullOrWhiteSpace(from) ? from : "0";
    var rangeEnd = !string.IsNullOrWhiteSpace(to) ? $", stop: {to}" :
string.Empty;
    var aggregationFunction = !string.IsNullOrWhiteSpace(agrFn) ? agrFn :
"mean";
    var stepClause = !string.IsNullOrWhiteSpace(step)
```

```
        ? $"|> aggregateWindow(every: {step}, fn: {aggregationFunction},
createEmpty: false) |> sort(columns: [\"_time\"])"
        : string.Empty;

    var query = new StringBuilder()
        .Append($"from(bucket:\"{bucketId}\")")
        .Append($" |> range(start: {rangeStart}{rangeEnd})")
        .Append($" |> filter(fn: (r) => r._measurement ==
\"{measurementId}\")")
        .Append(stepClause);

    var queryApi = _influxClient.GetQueryApi();
    var tables = await queryApi.QueryAsync(query.ToString(),
TimeSeriesSettings.InfluxOrgId);

    return tables;
}
```

Листинг 4 – Пример сбора данных в один JSON-файл

```
private List<TimeSeriesDataJsonEntity>
CreateTimeSeriesDataJsonEntities(List<FluxTable> measurementsResult)
{
    var timeseriesDataJsonEntities = new List<TimeSeriesDataJsonEntity>();
    foreach (var table in measurementsResult)
    {
        foreach (var record in table.Records)
        {
            var tags = record.Values
                .Where(x => !x.Key.StartsWith('_') && x.Key != "result" &&
x.Key != "table")
                .ToDictionary(x => x.Key, x => x.Value.ToString());

            var timestamp = record.GetTimeInDateTime().ToString();

            var timeseriesDataJsonEntity = timeseriesDataJsonEntities
                .Find(td => td.Timestamp == timestamp &&
td.Tags.SequenceEqual<KeyValuePair<string, string>>(tags));

            if (timeseriesDataJsonEntity == null)
            {
                var fields = new Dictionary<string, object> { {
record.GetField(), record.GetValue() } };

                timeseriesDataJsonEntities.Add(new TimeSeriesDataJsonEntity
                {
                    Timestamp = timestamp,
                    Fields = fields,
                    Tags = tags
                });
            }
            else
            {
                timeseriesDataJsonEntity.Fields[record.GetField()] =
record.GetValue();
            }
        }
    }
    return timeseriesDataJsonEntities;
}
```

ПРИЛОЖЕНИЕ Б

Листинг 5 – Извлечение данных из CSV-файла

```
public async Task<(int columnsCount, int rowCount, HashSet<string> columns,
Dictionary<string, HashSet<string>> tags)> UploadDataFromCsv(string
measurementId, string bucketId, List<string> lines)
{
    await DeleteMeasurementById(measurementId, bucketId);

    string[] headers = lines[0].Split(',');
    PointData[] influxRows = new PointData[lines.Count - 1];

    var columnsCount = headers.Length - 1;
    var rowCount = lines.Count - 1;
    var columns = new HashSet<string>();
    var tags = ExtractTags(headers);

    for (int i = 1; i < lines.Count; i++)
    {
        influxRows[i - 1] = CreateRow(headers, lines[i], measurementId,
tags);
    }

    await _influxClient.GetWriteApiAsync().WritePointsAsync(influxRows,
bucketId, TimeSeriesSettings.InfluxOrgId);

    return (columnsCount, rowCount, columns, tags);
}

private PointData CreateRow(string[] headers, string line, string
measurementId, Dictionary<string, HashSet<string>> tags)
{
    string[] rowValues = line.Split(',');
    PointData row = PointData.Measurement(measurementId)

.Timestamp(DateTime.FromFileTimeUtc(DateTime.Parse(rowValues[0]).ToFileTimeU
tc()), WritePrecision.Ns);

    for (int j = 1; j < headers.Length; j++)
    {
        if (headers[j].StartsWith('_'))
        {
            row = row.Tag(headers[j].Substring(1), rowValues[j]);
            tags[headers[j][1..]].Add(rowValues[j]);
        }
        else
        {
            row = row.Field(headers[j], Double.Parse(rowValues[j]));
        }
    }
    return row;
}
```

Листинг 6 – Извлечение данных из JTS-файла

```
private List<PointData> CreateInfluxRows(TimeSeriesJTSEntity jsonTsData,
string measurementId, HashSet<string> columns, Dictionary<string,
HashSet<string>> tags)
{
```

```
    var influxRows = new List<PointData>();

    foreach (var item in jsonTsData.Data)
    {
        var row = PointData.Measurement(measurementId)

.Timestamp(DateTime.FromFileTimeUtc(DateTime.Parse(item.Ts.ToString()).ToFil
eTimeUtc()), WritePrecision.Ns);

        foreach (var column in jsonTsData.Header.Columns)
        {
            string columnName = column.Key;
            var columnData = item.F[columnName];

            if (column.Value.Name.StartsWith("_") &&
!tags.ContainsKey(column.Value.Name))
            {
                tags[column.Value.Name[1..]] = new HashSet<string>();
            }
            else
            {
                columns.Add(column.Value.Name);
            }

            if (columnData != null)
            {
                if
(Double.TryParse((ReadOnlySpan<char>)columnData.V.ToString(),
NumberStyles.Any, CultureInfo.InvariantCulture, out double value))
                {
                    row = AddFieldOrTagToRow(row, column, value, tags);
                }
            }
        }

        influxRows.Add(row);
    }

    return influxRows;
}
```

Листинг 7 – Контроллер для веб-сокета

```
[Route("{timeSeriesMetadataId}/data/ws")]
public async Task GetFromWebSocket([FromRoute] string timeSeriesMetadataId)
{
    var timeSeriesMetadata =
_timeSeriesMetadataService.GetTimeSeriesMetadata(timeSeriesMetadataId);
    var bucketMetadata =
_bucketMetadataService.GetBucketMetadata(timeSeriesMetadata?.BucketId.ToStri
ng());

    if (IsInvalidMetadata(timeSeriesMetadata, bucketMetadata))
    {
        HttpContext.Response.StatusCode = StatusCodes.Status404NotFound;
        return;
    }

    if (!HttpContext.WebSockets.IsWebSocketRequest)
    {
        HttpContext.Response.StatusCode = StatusCodes.Status400BadRequest;
```

```
        return;
    }

    using var webSocket = await
HttpContext.WebSockets.AcceptWebSocketAsync();

    try
    {
        WebSocketReceiveResult result;
        do
        {
            var buffer = new byte[1024 * 4];
            result = await webSocket.ReceiveAsync(new
ArraySegment<byte>(buffer), CancellationToken.None);
            var message = Encoding.UTF8.GetString(buffer, 0, result.Count);

            var timeSeriesEntity =
JsonConvert.DeserializeObject<TimeSeriesWebSocketEntity>(message);
            var uploadResult = await
_timeSeriesService.UploadDataFromWebSocket(timeSeriesMetadata.Id.ToString(),
bucketMetadata.InfluxId, timeSeriesEntity);


_timeSeriesMetadataService.UpdateTimeSeriesMetadata(timeSeriesMetadata.Id.To
String(), timeSeriesMetadata.Name, timeSeriesMetadata.Description,
uploadResult.rowCount, uploadResult.columnsCount, uploadResult.columns,
uploadResult.tags);
        }
        while (!result.CloseStatus.HasValue);
    }
    catch (WebSocketException ex)
    {
        Console.WriteLine(ex.Message);
    }

      await webSocket.CloseAsync(result.CloseStatus.Value,
result.CloseStatusDescription, CancellationToken.None);
}
```

ПРИЛОЖЕНИЕ В

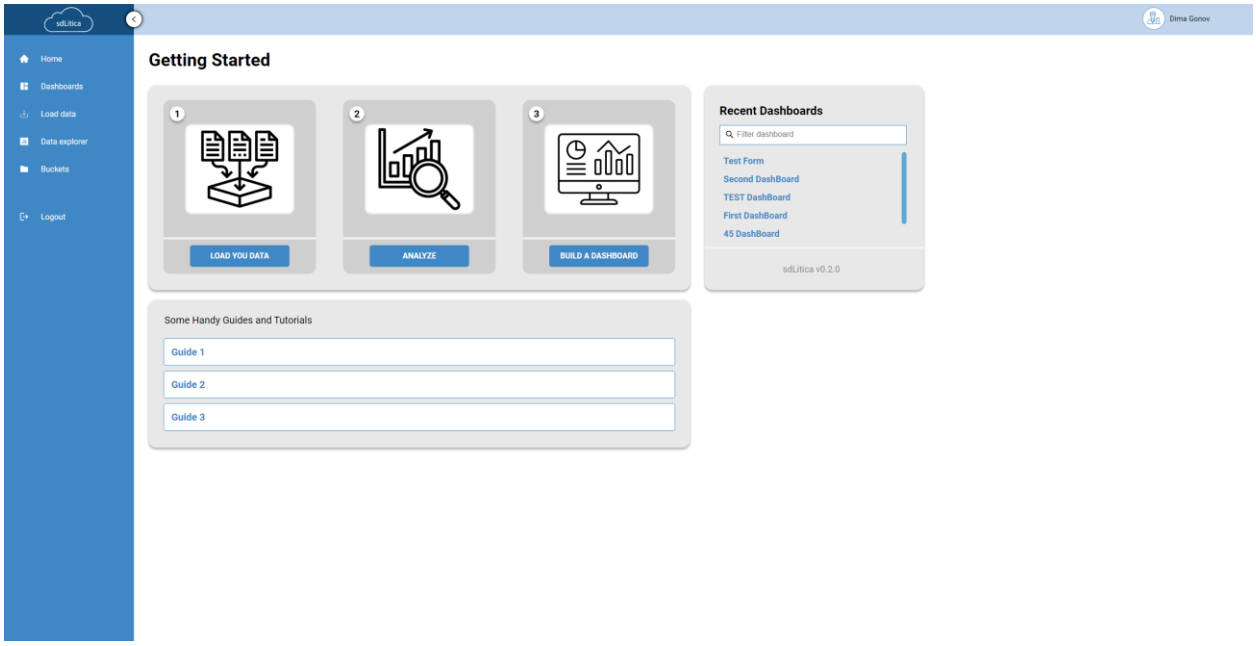


Рисунок 8 – Главная страница

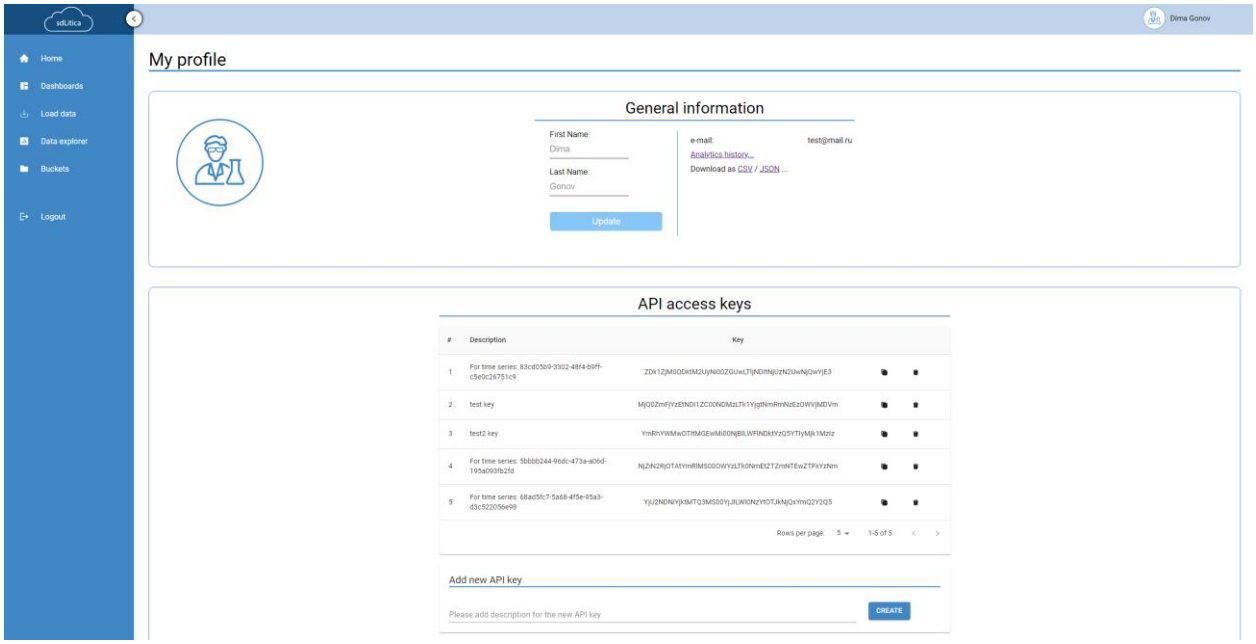


Рисунок 9 – Страница личного кабинета пользователя

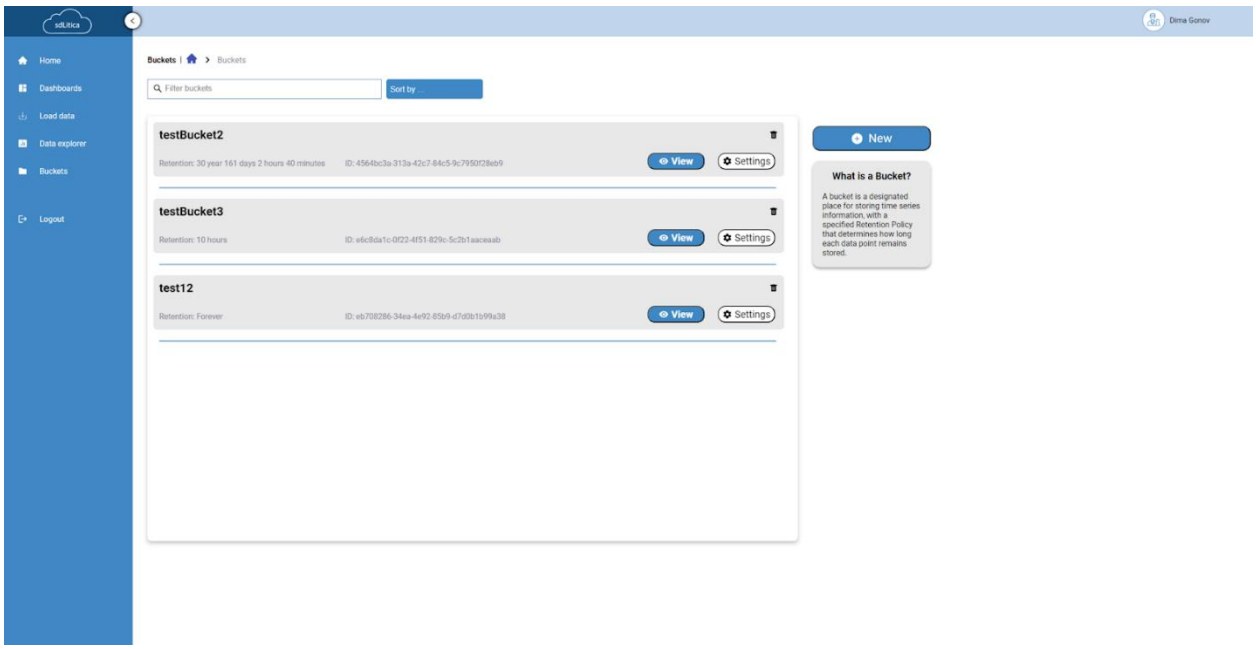


Рисунок 10 – Страница хранилищ

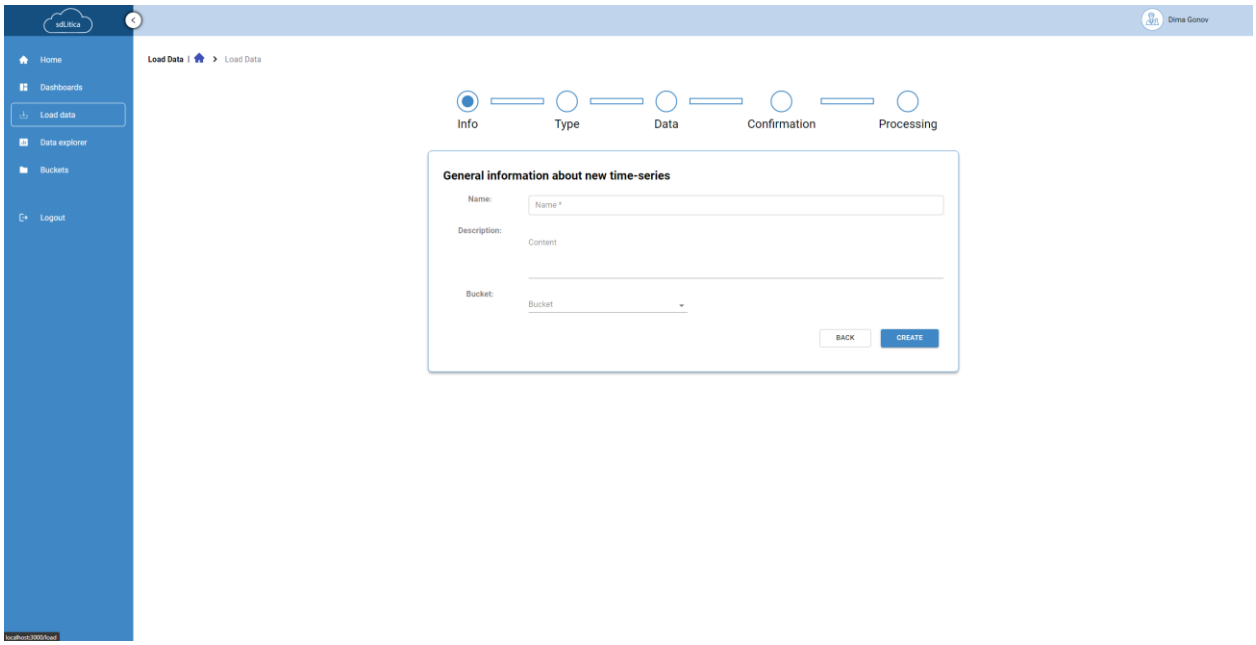


Рисунок 11 – Страница загрузки данных (информация о временном ряде)

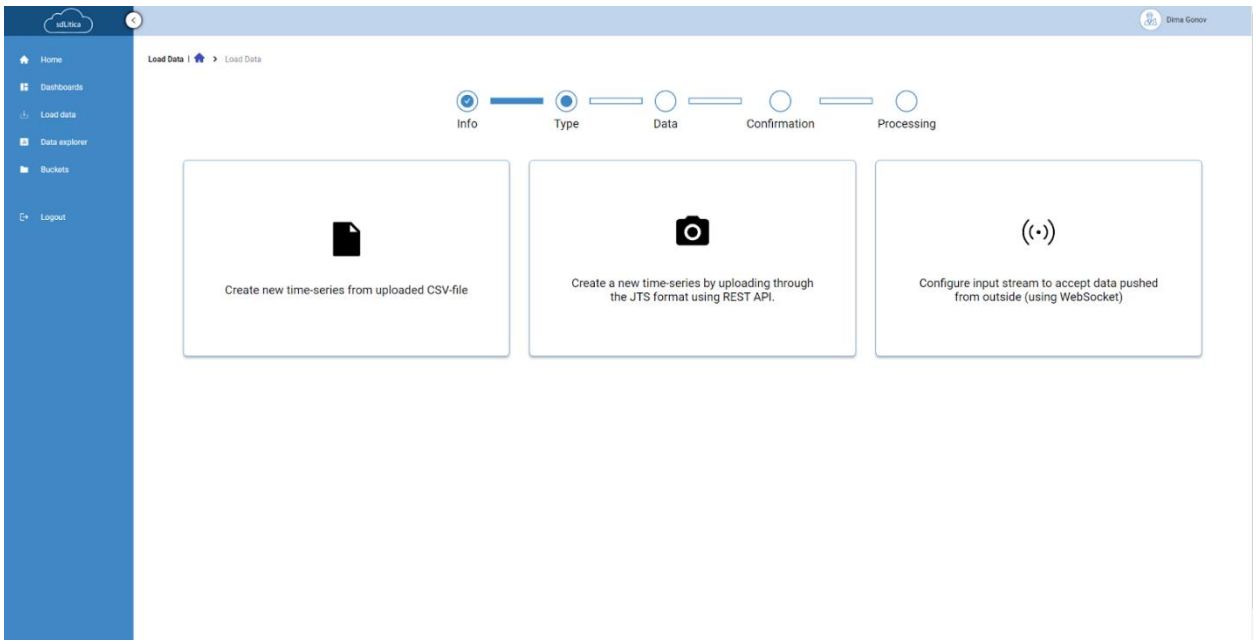


Рисунок 12 – Страница загрузки данных (выбор способа загрузки)

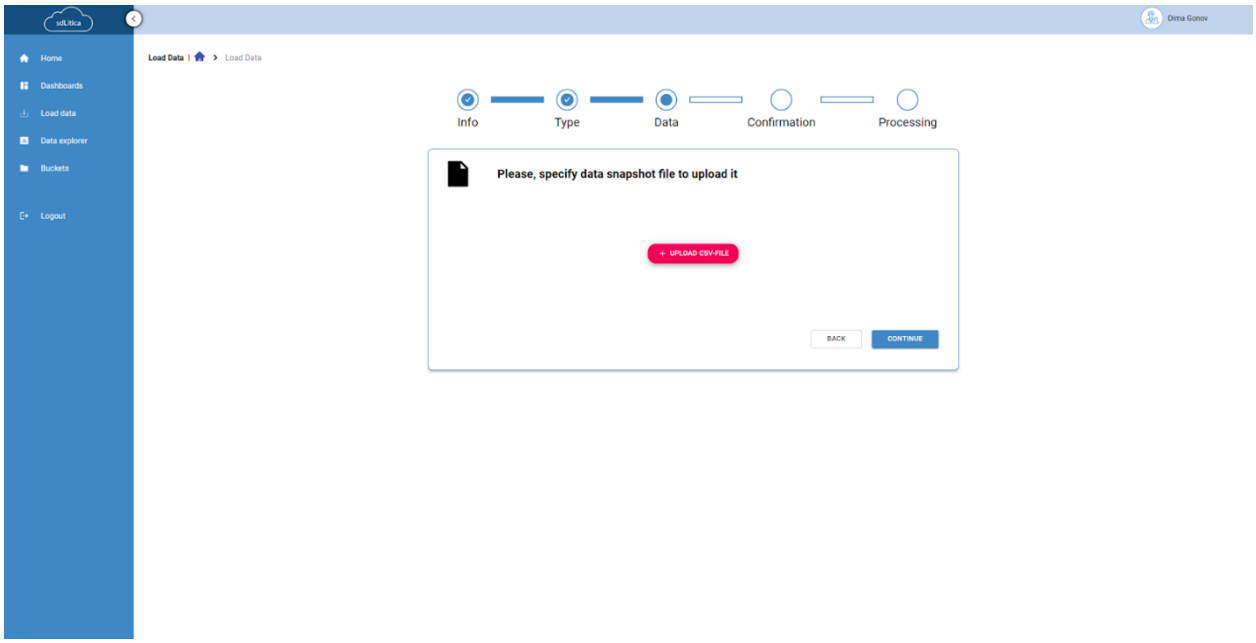


Рисунок 13 – Страница загрузки данных (получение данных из файла)

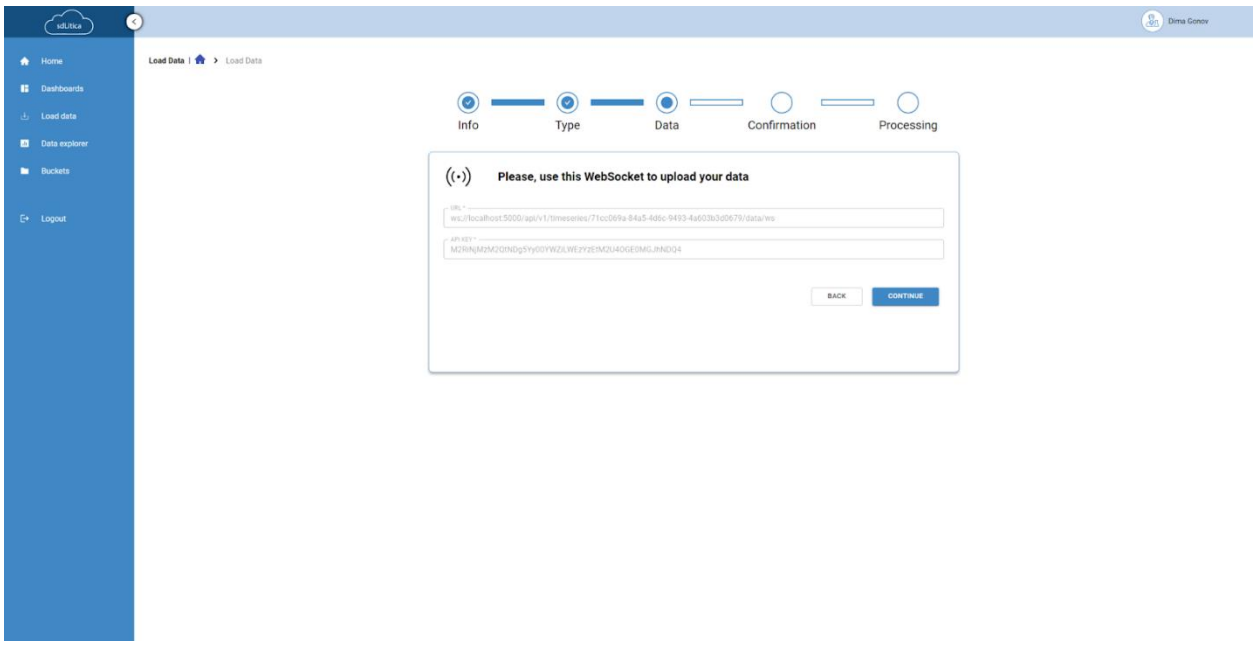


Рисунок 14 – Страница загрузки данных (с помощью WebSocket)

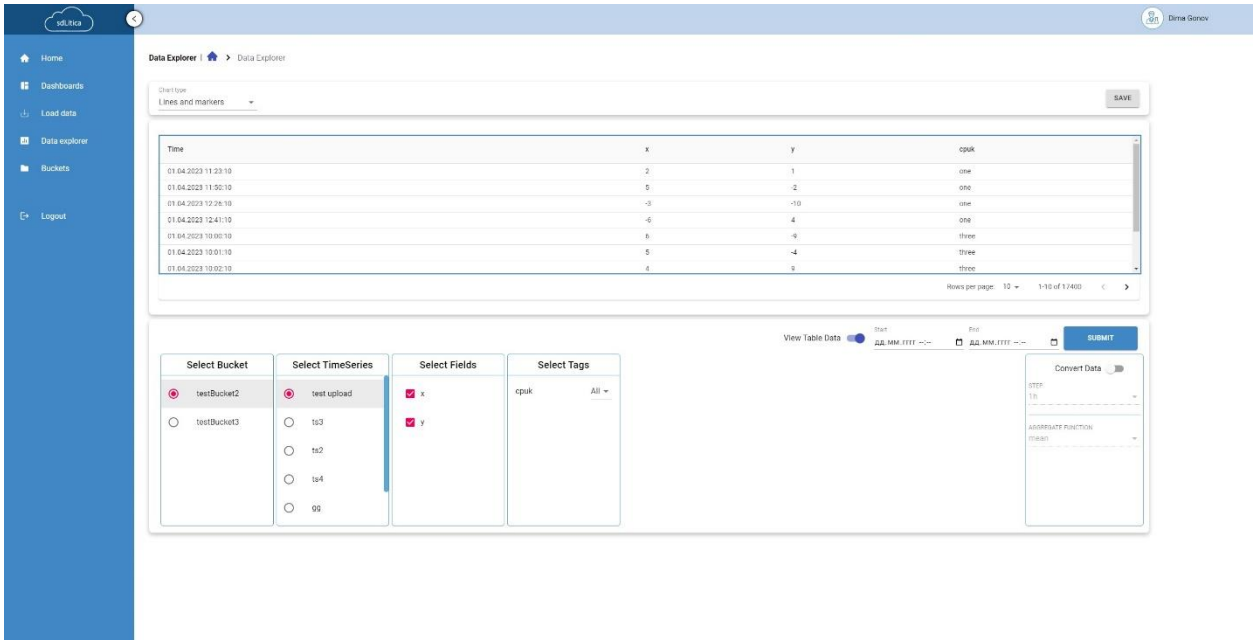


Рисунок 15 – Пример визуализации данных в виде таблицы



Рисунок 16 – Визуализация временного ряда с помощью графиков (линия и точки)

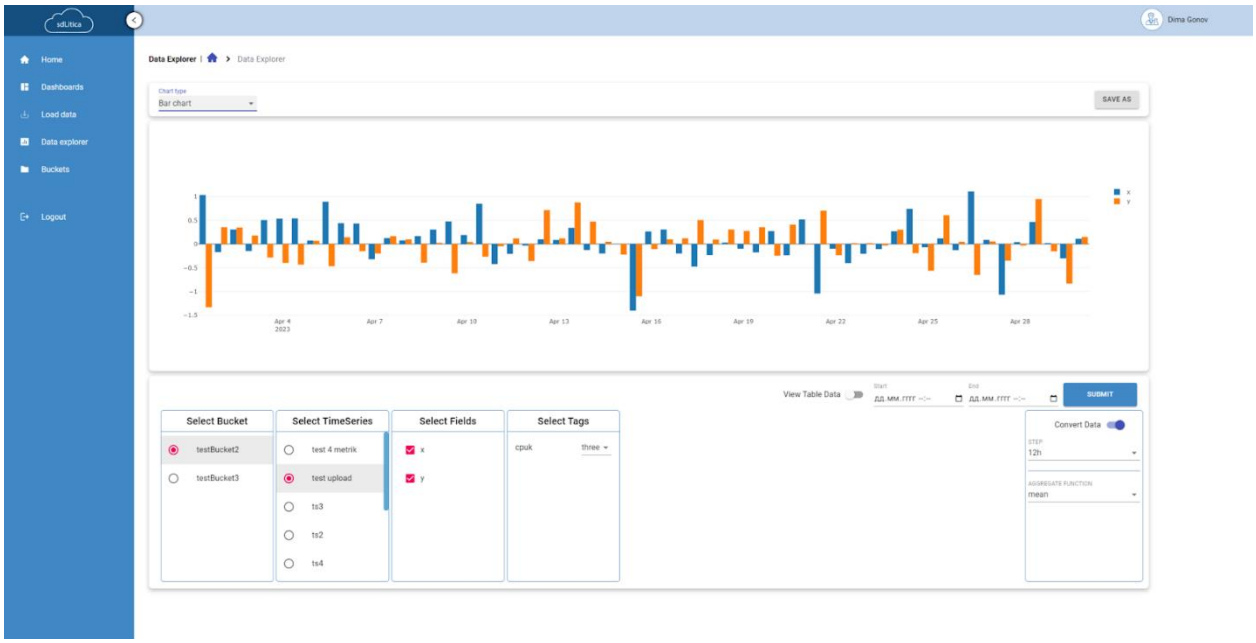


Рисунок 17 – Визуализация временного ряда с помощью графиков (гистограмма)

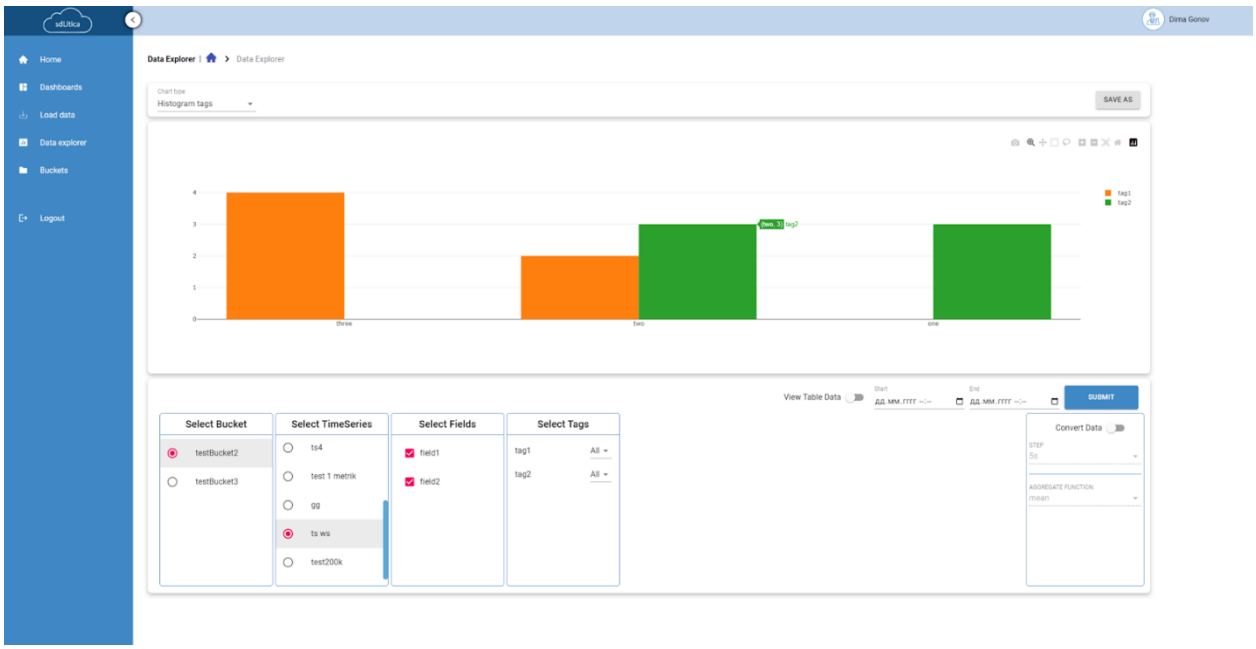


Рисунок 18 – Визуализация временного ряда в виде гистограммы тегов

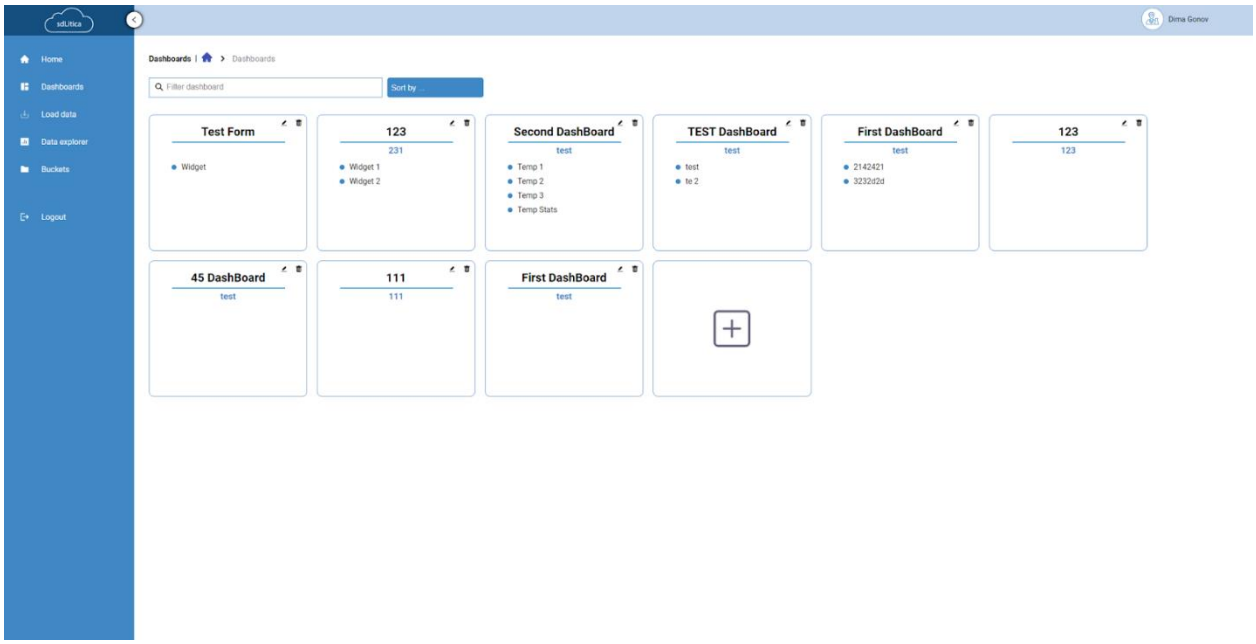


Рисунок 19 – Страница списка карточек приборных панелей